Oppimispäiväkirja: R-kielen perusteet, datankäsittely ja tilastotestit

Aaro Pohjalainen

2026-03-05

Table of contents

Oppimispäiväkirjan esittely 6

I Tutkimusmenetelmät 1 - oppimispäiväkirja 7

1 Kuvaavat tunnusluvut, varianssi ja keskihajonta 9

II R-kurssin tehtävät ja muistiinpanot 10

2 Osa 1 - Quarto-kirja 11
- 2.1 Lähteiden merkkaaminen ja viittaus quarto kirjassa . . . 11
- 2.2 **Video 4: Quarto kirjan luominen** . . . 12

3 Osa 2 - R-ohjelmoinnin perusteet 17
- 3.1 Apply, quantile & aggregate . . . 17
  - 3.1.1 Teoriaa kvantiileista . . . 18
  - 3.1.2 Pylväsdiagrammin piirtäminen . . . 21
  - 3.1.3 Dataframen tallentaminen .csv muotoon . . . 21
- 3.2 Luento 2 . . . 22
  - 3.2.1 **2. osan datansyöttötehtävä** . . . 23
- 3.3 Funktiot: readline(), paste(), data.frame(), for loop(){}, as.numeric(), numeric(), character(), matrix() . . . 24

4 Osa 3 - Raja-arvolause ja permutaatiotesti 27
- 4.1 Osa 3. alkutehtävät . . . 27
- 4.2 Permutaatiotesti . . . 29
  - 4.2.1 Teoriaa permutaatiosta . . . 29
  - 4.2.2 Permutaatiotesti ja bootstrap . . . 30

5 Osa 4 - Tekoälyn käyttö ohjelmoinnin tukena 36
- 5.1 Ohjelmointia tekoälyn avustamana, luento 26.03 . . . 36
  - 5.1.1 AI:n käyttö . . . 36
  - 5.1.2 AI-työkaluja . . . 36
  - 5.1.3 R-koodin tuottaminen . . . 36
  - 5.1.4 Skriptien optimointi ja organisointi . . . 37

5.2 PreProsessointi funktio tehtävä . . . 38
5.3 Visualisointitehtävä . . . 42

6 Osa5 - Versionhallinta, Anova ja Regressio **49**
6.1 Luento 09.04: Versionhallinta, organisointi, kommentointi . . . 49
6.1.1 Versionhallinta . . . 49
6.2 Luento 16.04: Anova & Regressio . . . 50
6.2.1 Tilastotestit . . . 50
6.2.2 1-suuntainen anova . . . 52
6.2.3 Faktoriaalinen anova . . . 52
6.2.4 Useampisuuntainen anova . . . 53
6.2.5 ANCOVA . . . 53
6.2.6 ANOVAN oletukset . . . 53
6.2.7 ToistomittausANOVA . . . 54
6.2.8 Puuttuvat arvot Anovassa . . . 55
6.2.9 Post Hoc testit ja kontrastit . . . 55
6.2.10 Anova vs GLM . . . 55
6.2.11 Mallit Anova & lineaarinen regressio . . . 56
6.3 Toistettava ja avoin tiede (luentovideo) . . . 57
6.4 Erilaisia aineistoja ja niiden erityispiirteitä (luentovideo) . . . 57
6.5 Replicate funktio for-loopin sijaan (luentovideo) . . . 59
6.6 Skriptistä funktio (luentovideo) . . . 60
6.7 Skriptien ja funktioiden kuvaus (luentovideo) . . . 60
6.8 Matikka-aineiston käsittelytehtävä . . . 60
6.8.1 Skripti 1 . . . 61
6.8.2 Skripti 2 . . . 68

III R-komentoja **69**

7 Kuvaajien piirtäminen **71**
7.1 Histogrammi . . . 71
7.2 Pylväsdiagrammi . . . 72
7.3 Plot . . . 73

8 ggplot paketti **75**
8.1 Pylväsdiagrammi ggplotilla . . . 75

9 Datasetin luominen **78**
9.1 Datan avaaminen . . . 78
9.2 Datasetin kuvailu . . . 78
9.3 rnorm() . . . 80

9.4 set.seed() funktio . . . 81
9.4.1 Havainnollistava esimerkki . . . 81
9.5 Data framen luominen . . . 82
9.6 Satunnaistaminen tai satunnaisnumero sample() funktiolla . . . 83
9.7 Uuden kansion luominen dir.create() . . . 84
9.8 Kansion luominen if-lauseen kanssa . . . 84
9.9 Tiedosto ja kansiopolun muuttaminen file.rename() . . . 85
9.10 Tiedostopolun kirjoittaminen file.path() funktiolla . . . 86
9.11 Tiedostojen hakeminen ylä- tai alakansioista . . . 86

10 Tilastotestejä **87**
10.1 T-testi . . . 87
10.1.1 T-testi syöttämällä suoraan kaksi ryhmää . . . 87
10.1.2 T-testi kaavamuodossa (tällöin voidaan hakea tieto dataframesta) . . . 88
10.2 Korrelaatio . . . 88

11 Matemaattisia funktioita **90**
11.1 Kertoma: factorial() . . . 90
11.2 Kombinaatiot: choose() . . . 90
11.3 Vektorien laskutoimitukset . . . 91
11.4 Summafunktiot: sum(), colSums() ja rowSums() . . . 92
11.5 Matemaattisia operaattoreita . . . 93

12 Tekstikomentoja **94**
12.1 Tekstin kirjoittaminen ja .txt tekstitiedoston luominen . . . 94
12.2 Merkkijonojen yhdistäminen Paste() funktiolla . . . 94
12.3 PDF tiedoston luominen ja siihen asioiden kirjoittaminen . . . 95

13 Varmuuskopiointi **99**
13.1 Git ja github . . . 99
13.2 .gitignore . . . 100

14 Indeksointi / subsetting **101**
14.1 Numeerinen indeksointi . . . 101
14.2 Looginen indeksointi . . . 103
14.3 Kaksiulotteinen indeksi . . . 104
14.4 Yhdet vs. kahdet hakasulkeet . . . 104

15 Funktion luominen ja kutsuminen **105**
15.1 Funktion luominen: function() . . . 105
15.1.1 Perusrakenne . . . 105
15.1.2 Argumentit . . . 106

16 Sekalaista **107**
16.1 Run-napin luominen quarto-kirjaan . . . 107
16.2 Piilotettava / foldattava koodilohko quarto-kirjaan . . . 108
16.3 | ja & merkit . . . 108

IV Tilastotiede **109**

17 Poikkeavat havainnot **111**
17.1 MAD-arvo . . . 111
17.1.1 MAD-arvon laskeminen R:ssä . . . 112
17.2 Poikkeavien havaintojen poistaminen . . . 112

18 Normaalisuuden testaaminen **115**
18.1 Kolmogorov-smirnov normaalisuustesti ks.test() . . . 115
18.2 Shapiro-wilk normaalisuustesti shapiro.test() . . . 115
18.3 Normaalijakauma . . . 115

19 Matematiikan laskusääntöjä **116**
19.1 Kymmenpotenssi (e) . . . 116

20 Summary **117**

References **118**

Oppimispäiväkirjan esittely

Kirja sisältää

Suosittelen lukemaan kirjaa verkkosivuversiona osoitteesta: https://aaropoh.github.io/LearningDiary/

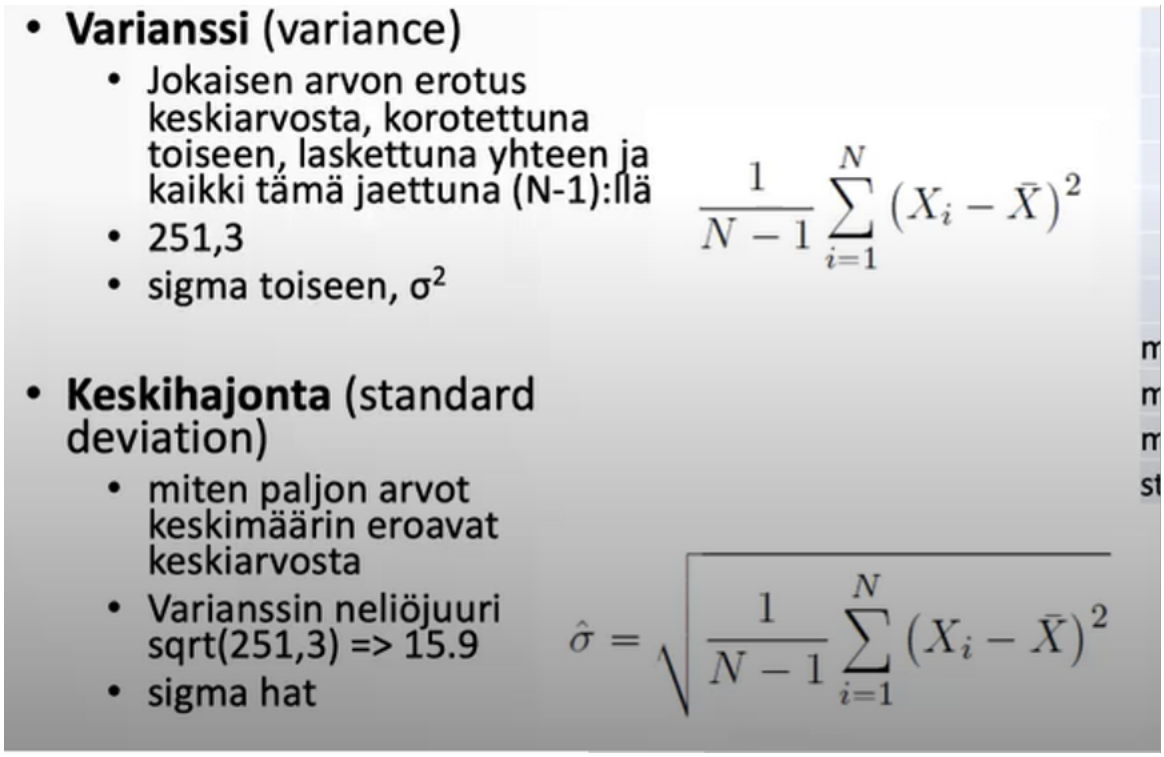
Part I

Tutkimusmenetelmät 1 - oppimispäiväkirja

Tähän oppimispäiväkirjaan lisään etenkin hyödylliseksi kokemiani muistiinpanoja Jamovin ja r-

kielen käytöstä ja opiskelusta. Lisäksi saatan kommentoida, mitkä asiat tuntuivat vaikeilta oppia ja mihin täytyy perehtyä tarkemmin myöhemmin. Pidän pääpainon kuitenkin omasta mielestä hyödyllisissä muistiinpanoissa, joita uskon voivani tarvita myöhemminkin. Iso osa muistiinpanoista on kuvankaappauksia luentodioista, joihin olen sitten lisännyt omia huomioita tai asiaa, joita videolla on kerrottu.

1 Kuvaavat tunnusluvut, varianssi ja keskihajonta



Varianssi on siis keskihajonta korotettuna toiseen. Salmela (2026)

Part II

R-kurssin tehtävät ja muistiinpanot

2 Osa 1 - Quarto-kirja

R-ohjelmointikielen kurssin ensimmäisen viikon muistiinpanot ja tehtävät.

2.1 Lähteiden merkkaaminen ja viittaus quarto kirjassa

Merkataan references.bib tiedostoon seuraavalla tyylillä

```
@book{thulin2024modern,
  title={Modern Statistics with R: From wrangling and exploring data to inference and predic
  author={Thulin, M{\aa}ns},
  year={2024},
  publisher={Chapman and Hall/CRC},
  url = {https://www.modernstatisticswithr.com}
```

Aiemmin opittua tästä kirjasta Navarro and Foxcroft (2025)

See Thulin (2024) for additional discussion of literate programming.

kokeilu Speekenbrink (2023)

Jos on useampi kirjoittaja, niin kirjoittajat erotellaan and sanalla tähän tyyliin

```
@article{Rmoniste,
  title={R-tutuksi [oppimateriaali]},
  author={Hyvönen, Ville and Karttunen, Henri and Lehtonen, Toni and Leivonen, Aku and Virola
  year={2018},
  publisher={Helsingin yliopisto, Matematiikan ja tilastotieteen laitos}
```

BibTeX saattaa muuttaa otsikon kaikki kirjaimet pieniksi (paitsi ensimmäisen). Jos haluat säilyttää esimerkiksi sanan "R" isona tai pitää "[oppimateriaali]"-tekstin juuri sellaisenaan, se kannattaa suojata aaltosulkeilla.

```bibtex
@manual{Rmoniste,
  title = {{R}-tutuksi [{O}ppimateriaali]},
  author = {Hyvönen, Ville and Karttunen, Henri and Lehtonen, Toni and Leivonen, Aku and Vir
  year = {2018},
  organization = {Helsingin yliopisto, Matematiikan ja tilastotieteen laitos},
  note = {Luettavissa verkossa},
  url = {https://www.mv.helsinki.fi/home/mjxpirin/medstat_course/material/Rmoniste_tiltu1.pd
}
```

2.2 Video 4: Quarto kirjan luominen

tehdään kirja / nettisivusto, josta voi aina nopeasti etsiä apua

- quarto dokumentit ovat .qmd tiedostoja
- Control + L —> tyhjentää konsolin
- quarto install tinytex
  - jotta quarto osaa tehdä pdf:iä täytyy asentaa tinytex
  - voi tehdä myös terminaalissa
  - vielä epäselvää mikä on konsolin ja terminaalin ero ja miksi nämä komennot tehtiin nimenomaan terminaalissa

```
#quarto render
```

- komento tekee html kirjan

```
#quarto render --to pdf
```

- (eli kaksi viivaa ja to pdf quarto renderin perään)
- tekee pdf:n projektista

```
#quarto preview
```

- tekee html version joka toimii local host nettisivulla

Kaikki yllä olevat komennot tehtiin terminaalissa

YML tiedosto

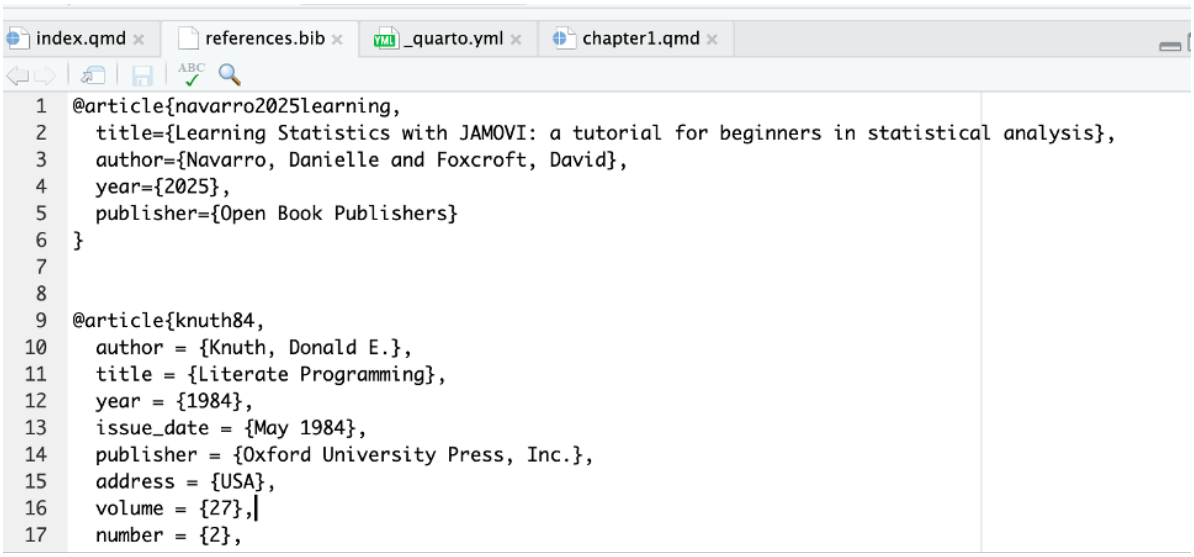
- yml fileen voidaan tehdä muokkuksia ja esim. lisätä kirjan kappaleita

- yml tiedostoon siis riittää, että listataan tiedoston nimi
- qmd tiedostoon voidaan sitten kirjoittaa mitä halutaan
- esim. tehtiin chapter1.qmd tiedosto

Quarto kirja

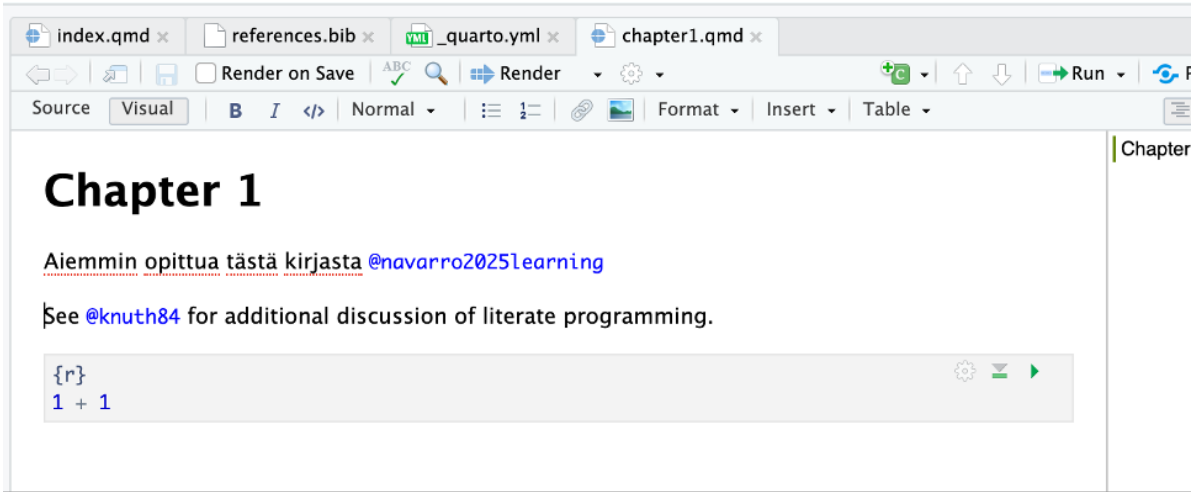
- yml tiedoston chapters kohtaan listataan kappaleeet

. jokaisesta kappaleesta täytyy olla oma .qmd tiedosto

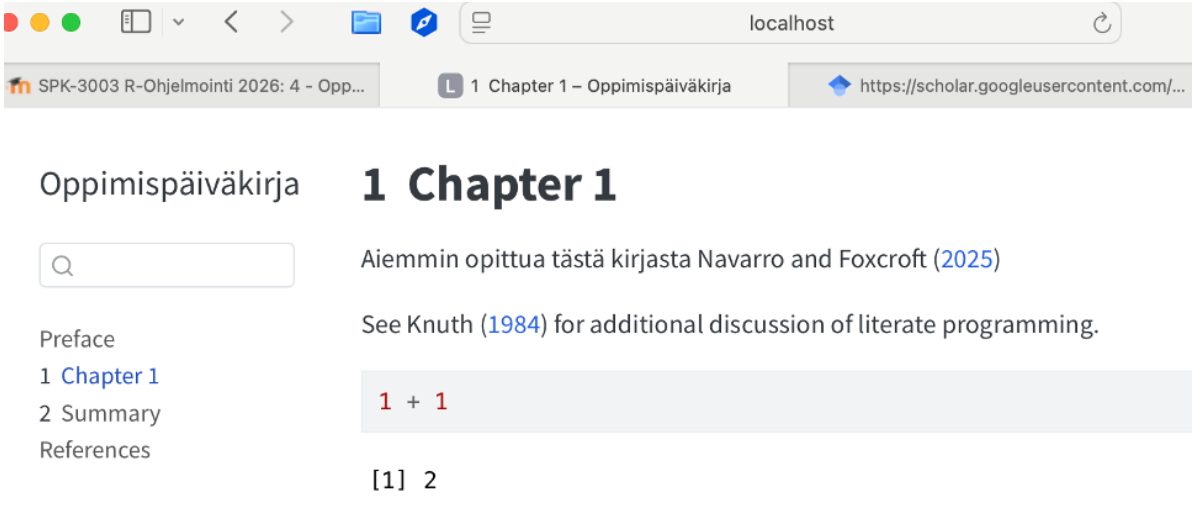


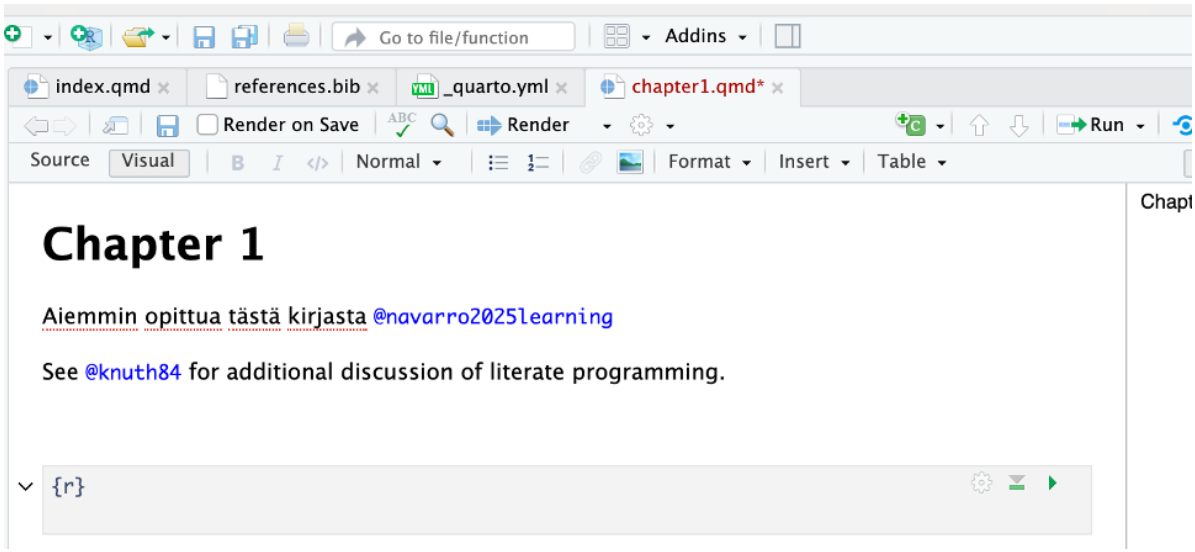
references.bib

- tiedostoon voidaan lisätä lähteitä bib text muodossa



- lähteisiin voidaan viitata @ merkillä ja nimellä, jolloin ne näkyvät näin:





- yläoikealla olevasta vihreästä c kuvakkeesta voidaan lisätä koodinpätkiä
  - se lisää alhaalla näkyvän harmaan laatikon, johon voi kirjoittaa r-koodia (voi valita muitakin ohjelmointikieliä)

```
1 + 1
```

```
[1] 2
```

```r
2 * 2
```

```
[1] 4
```

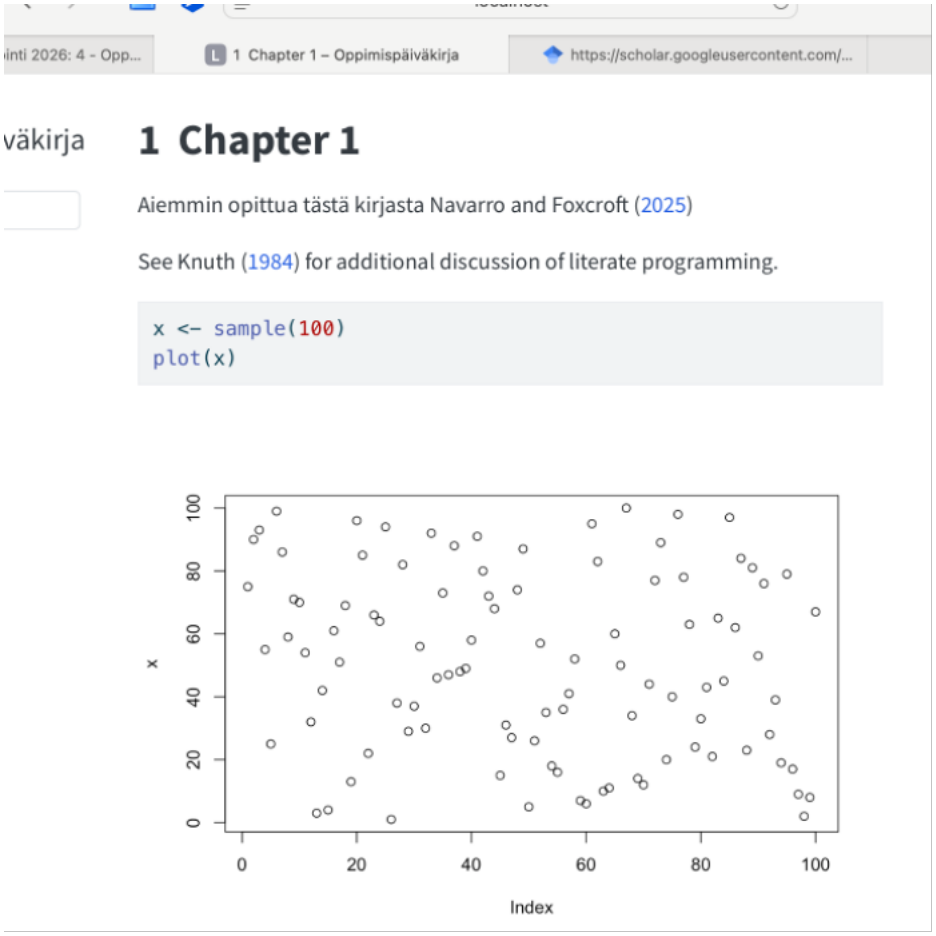
```r
4 / 5
```

```
[1] 0.8
```

- Lisätyt koodinpätkät näyttävät tältä.
- Jos haluaa koodinpätkän, jota ei suoriteta kannattaa käyttää komentoa
  - #| eval: false

```r
install.packages(c("swirl", "swirlify"))
library(swirl)
library(swirlify)
swirl::install_course("R Programming")
swirl::install_course("The R Programming Environment")
swirl::install_course("Advanced R Programming")
swirl()
```

- Nyt boksiin laitettua koodinpätkää ei suoriteta, mutta näyttää kuitenkin hyvältä ja sen voi suoraan kopioida
- Yllä näkyvä swirl paketti on R-kielen opetuspaketti. Sen avulla voi myös opetella kieltä.



- koodi näkyy kirjassa yllä olevalla tavalla
  - harmaan koodipalkin oikeasta reunasta voi kopsata koodin

3 Osa 2 - R-ohjelmoinnin perusteet

R-ohjelmointikielen kurssin toisen viikon muistiinpanot ja tehtävät.

3.1 Apply, quantile & aggregate

Ensiksi luotiin datafreimi data.frame() funktiolla

```
data <- data.frame(
  Group = as.factor(rep(c("GroupA","GroupB","GroupC"),10, each=10)),
  Age = as.numeric(sample(18:65, 300, replace=TRUE)),
  WM =  as.numeric(sample(1:100, 300, replace=TRUE))
)

aggregate(Age ~ Group, data = data, FUN = mean)
```

```
   Group   Age
1 GroupA 42.41
2 GroupB 43.04
3 GroupC 43.70
```

```
quantile(data$WM, probs = seq(0, 1, 0.9))
```

```
 0% 90%
  1  94
```

```
apply(data[,2:3], 2, mean)
```

```
     Age       WM
43.05000 55.10667
```

Aggregate funktiossa ensiksi valitaan numeerinen freimi datasetistä, josta keskiarvo (tai jokin muu) lasketaan, sitten ~ merkin jälkeen # ryhmittelevä tekijä, eli se minkä mukaan data jaotellaan. # data = kohtaan datasetti, # FUN = kohtaan käytettävä funktio, jota käytetään.

Quantile fuktiolla saadaan laskettua kvantiili. Quantile funktiossa ensiksi valitaan data, josta kvantiili otetaan. Kohtaan seq(, ,kolmanteen väliin kvantiili, joka halutaan)

Apply funktiolla voidaan käyttää haluttua funktiota datasetin valittuihin riveihin tai sarakkeisiin. Apply funktiossa ensiksi datasetti, [hakasulkeissa] voidaan indeksien avulla määritellä # mitkä rivit tai sarakkeet datasta otetaan käyttöön. Menee aina näin: [rivit, sarakkeet] Nyt valittiin [2:3] komennolla 2. ja 3. sarake datasta Ekan pilkun jälkeen laitetaan 1 tai 2. Jos laitetaan 1, niin silloin jaotellaan data rivien mukaan. Jos taas 2, niin se jaotellaan sarakkeiden mukaan.

3.1.1 Teoriaa kvantiileista

Kvantiilit / persentiilit

- x-Kvantiili kuvaa sitä pistettä jonka alle jää x prosenttiosuus ja ylle y prosenttiosuus havainnoista (y= 100%-x)
  - esim. mediaanin (50-persentiili) alle jää 50 % havainnoista ja ylle 50%
  - tai 90-persentiilin alle 90% ja ylle 10%
  - Esim.
    * Manuaalisesti saadaan haettua siten, että asetetaan luvut suuruusjärjestykseen ja etsitään oikea kohta. Esim 7 luvun sarjassa (1,2,3,4,5,6,7) mediaani on 4. Yläpuolelle jää 3 lukua ja alapuolelle 3.
    * 90.-persentiili sarjassa (1,2,3,4,5,6,7) asettuu lukujen 6 ja 7 välille. Kun luku ei osu suoraan kohdalle täytyy se interpoloida.
  - kvantiili kuvaa siis alueiden rajalla olevaa pistettä
- R-kielellä kvantiilin saa etsittyä quantile() funktiolla

```
quantile(c(1,2,3,4,5,6,7), probs = seq(0, 1, 0.5), na.rm = FALSE,
         names = TRUE)
```

```
  0%  50% 100%
   1    4    7
```

```
    quantile(x, prob = seq(0, 1, 0.5), na.rm = FALSE,
  names = TRUE, type = 7)
```

- x = data/vektori, josta kvantiili otetaan
- probs = seq(0, 1, 0.25) –> ekat kaksi numeroa pidetään aina 0 ja ykkösenä. Määrittää sekvenssin rajat. Viimeinen luku määrittää miten sekvenssi jaetaan
  - toisin sanottonu 0,25 kohtaan laitetaan haluttu kvantiili
    - 0,25 niin tulee persentiilit 25% välein
    - 0,90 tulee 90-persentiili ja 0,5 mediaani
- na.rm = FALSE, saa tarvittaessa puuttuvat arvot jätettyä huomiotta. Tällöin merkattava TRUE, jolloin poistaa puuttuvat arvot.
- names = TRUE, näyttää prosentit
- type =, valitaan tyyppi, toimii ilmankin.

seq()

```
seq(0,10,3)
```

```
[1] 0 3 6 9
```

```
#sekvenssi, 1. numero kertoo alkupisteen, toinen loppupisteen ja 3. numero
#   sen millä välein sekvenssi jatkuu eli numerot lisätään.
```

```
rep(1:4,times =3)
```

```
 [1] 1 2 3 4 1 2 3 4 1 2 3 4
```

```
#Repeat a vector ; 1. mitä toistetaan, 2. montako kertaa voi kirjoittaa times =, tai pelkästä


rep(1:3, each=5)
```

```
 [1] 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
```

```
#Repeat elements of a vector: 1. mitä toistetaan 2. each =, komennolla toistetaan jokainen el
```

list()

```r
l <- list(x = 1:5, y = c('a', 'b'), z = rep(1:4,times =3))
l
```

```
$x
[1] 1 2 3 4 5

$y
[1] "a" "b"

$z
 [1] 1 2 3 4 1 2 3 4 1 2 3 4
```

list() funktio tekee annetuista syötteistä listat. Voivat olla erilaisia ja eri mittaisia.

```r
my_list <- list(a = 1, b = 2, c = 3)
my_list$d <- 4
my_list
```

```
$a
[1] 1

$b
[1] 2

$c
[1] 3

$d
[1] 4
```

Lisätään listaan uusi alkio (**viljamin?**) tyyli

```r
lista <- list(x = 1:5, y = 6:10)
lista <- c(lista, list(z = 50:55))
lista
```

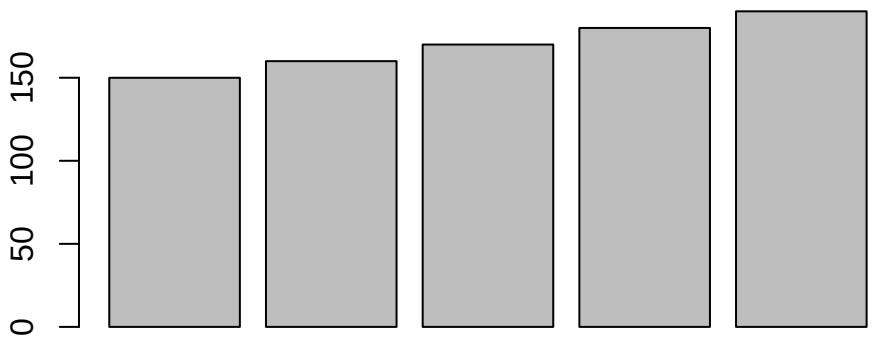
```
$x
[1] 1 2 3 4 5

$y
```

```
[1]  6  7  8  9 10

$z
[1] 50 51 52 53 54 55
```

Toinen mahdollinen tyyli lisätä alkio listaan.

3.1.2 Pylväsdiagrammin piirtäminen

```r
heights_var <- c(150, 160, 170, 180, 190)
barplot(heights_var)
```



3.1.3 Dataframen tallentaminen .csv muotoon

```r
df <- data.frame(name = c("John", "Jane", "Doe"), age = c(23, 25, 28))
write.csv(df, "new_data.csv")
```

- write.csv() komennolla: ekaan väliin tallennettava data, tokaan väliin luotavan tiedoston nimi

- write.csv() luomassa tiedostossa piste (.) toimii desimaalien erottajana ja pilkku (,) sarakkeiden erottelijana.
- write.csv2() käyttää suomalaistyyliin pilkkua (,) desimaalierottelijana ja puolipistettä (;) sarakkeiden erottelijana
  - voi olla hyödyllinen, kun halutaan viedä suomenkieliseen exceliin.
  - tosin excel kysyy csv. tiedostoa tuotaessa, että millä se erotellaan

3.2 Luento 2

kysy havaintojen lukumäärä

```
nObs <- readline(prompt = "Anna havaintojen lukumäärä: ")
```

muodosta tyhjä vektori havaintojen arvojen tallentamista varten

```
MyVar <- numeric(nObs)
```

tee loop, jossa kysytään jokaisen havainnon arvo

```
for (i in 1:nObs) { MyVar[i] <- readline(prompt = paste('Anna havainnon', i, 'arvo: ')) }
```

talleta arvot datakehikkoon

```
data3 <- data.frame(Kurssipisteet = MyVar)
```

tulosta konsoliin

```
print(data3)
```

3.2.1 2. osan datansyöttötehtävä

Tehtävänanto:

Tee ohjelma, jolla voit syöttää muuttujalle arvot. Eli ohjelma ensin kysyy käyttäjältä kuinka monta havaintoa talletetaan ja sitten ohjelma kysyy jokaisen havainnon arvon. Ja nämä talletetaan data.frame:en. lopuksi skripti tulostaa datan. palauta R-skripti

Lisähaasteena voit tehdä niin, että ohjelma kysyy kuinka monta muuttujaa talletetaan, ja muuttujien nimet, ja sitten vuorotellen kaikille muuttujille havaintojen arvot.

Tarpeellisia funktiota voi olla: readline() , esim: readline(prompt = "Anna havaintojen määrä:") data.frame() , uuden data-kehikon luominen paste() , muuttujien yhdistäminen, esim. paste("Anna havainnon", i, "arvo:") for-loop, esim.

for (i in 1:num_obs) {
  ….
} names(data) , data-kehikon muuttujien nimet print()

Ratkaisu:

```
#kysy havaintojen lukumäärä
nObs <-as.numeric(readline(prompt = "Anna havaintojen lukumäärä: "))

#kysy muuttujien lukumäärä

MuutLkm <- as.numeric(readline(prompt = "Anna muuttujien lukumäärä: "))

#muuttujien nimet

MuutNimet <- character(MuutLkm)

for (i in 1:MuutLkm){

  MuutNimet[i] <- readline( prompt = paste('Anna muuttujan',i, 'nimi: '))
  }

#luodaan datafreimi

datafreimi <- data.frame(matrix(nrow = nObs, ncol = MuutLkm,))
colnames(datafreimi) <- MuutNimet

#lisätään havaintoarvot datafremiin muuttuja kerrallaan

for (i in 1:MuutLkm){
```

```
  for (j in 1:nObs){
    datafreimi[j,i] <- readline(prompt = paste("anna muuttujan", MuutNimet[i], "havainnon", j

  }

}

print(datafreimi)
```

3.3 Funktiot: readline(), paste(), data.frame(), for loop(){}, as.numeric(), numeric(), character(), matrix()

```
nObs <- as.numeric(readline(prompt = "Anna havaintojen lukumäärä: "))
```

readline()

- R-kielen komento, jonka avulla ohjelma pysähtyy ja odottaa, että käyttäjä kirjoittaa jotain näppäimistöllä konsoliin.
- Kun R kohtaa readline()-komennon, se ei jatka koodin suorittamista ennen kuin painat Enter. Kaikki, mitä kirjoitat ennen Enteriä, tallennetaan muuttujaan.
- Tärkeää muistaa: Se palauttaa aina tekstiä
  - Tämä on yleisin virhe readline()-funktion kanssa. Vaikka kirjoittaisit numeron (esim. 25), R tallentaa sen tekstinä ("25"), ei lukuna.
  - Jos haluat käyttää syötettä laskutoimituksissa, sinun täytyy muuttaa se numeroksi as.numeric()-funktiolla:

prompt =

- On tärkeää huomata, että R-kielessä prompt ei ole itsenäinen "komento" samalla tavalla kuin vaikkapa print(), vaan se on **argumentti** (eli lisäasetus), jota käytetään muiden funktioiden sisällä.
- Yleisimmin esiintyy readline()-funktion yhteydessä.
- Prompt tarkoittaa suomeksi **kehotetta**. Sen tehtävä on kertoa käyttäjälle, mitä hänen odotetaan tekevän. Se tulostaa tekstin konsoliin ja jättää kursorin odottamaan käyttäjän kirjoitusta.

  Ilman prompt-asetusta tietokone vain pysähtyisi odottamaan syötettä, ja käyttäjä olisi ihmeissään, miksi mitään ei tapahdu.

as.numeric()

- muuttaa syötteen tekstista numeeriseksi
- tämän myötä esimerkiksi laskutoimitukset onnistuvat

```
numeric(5)
```

```
[1] 0 0 0 0 0
```

numeric()

- Luo tyhjän numeerisen vektorin, jonka voi täyttää
- Sulkuihin voi määritellä montako paikkaa vektoriin tulee
  - esim. numeric(5) luo vektorin, jossa 5 nollaa eli 5 tyhjää paikkaa täytettäväksi
  - voi laittaa myös muuttujan sulkeisiin, jolloin paikkoja tulee muuttujan arvon verran

```
character(6)
```

```
[1] "" "" "" "" "" ""
```

character()

- Luo tyhjän tekstipohjaisen "vektorin" / listan, jossa ennalta määritelty määrä paikkoja
- esim character(6) vektorissa 6 paikkaa
- Sulkuihin voi laittaa myös muuttujan, jolloin paikkojen lukumäärä tulee muuttujan arvon mukaan esim. character(Muuttuja)

```
datafreimi <- data.frame(matrix(nrow = nObs, ncol = MuutLkm,))
colnames(datafreimi) <- MuutNimet
```

data.frame()

- luo dataframen
- **Sarakkeiden määrä** määräytyy siitä, kuinka monta vektoria annat syötteeksi.
- **Rivien määrä** määräytyy annettujen vektorien pituudesta.

```r
data.frame(Eka = c(3,4,5,2),
           Toka = seq(2,8,2),
           Kolmas = c("Matti", "Jesse", "Katja", "Joni")
           )
```

```
  Eka Toka Kolmas
1   3    2  Matti
2   4    4  Jesse
3   5    6  Katja
4   2    8   Joni
```

- Yllä olevaan frameen laitettiin kolme vektoria, jotka kaikki olivat pituudeltaan 4 alkiota, joten syntyi kolmen sarakkeen ja neljän rivin datafreimi.
- Sarakkeet pystyy nimeämään luomisvaiheessa (Eka, Toka, Kolmas) tai myöhemmin colnames() funktiolla

```r
df <- data.frame(matrix(nrow = 5, ncol = 5))
colnames(df) <- c("Ikä", "Pituus", "Paino", "Ekstroversio", "Avoimuus")
df
```

```
  Ikä Pituus Paino Ekstroversio Avoimuus
1  NA     NA    NA           NA       NA
2  NA     NA    NA           NA       NA
3  NA     NA    NA           NA       NA
4  NA     NA    NA           NA       NA
5  NA     NA    NA           NA       NA
```

- Halutessaan dataframeen voi valmiiksi määrittää pohjalle tietyn määrän rivejä ja sarakkeita matrix() funktion avulla
- matrix(nrow = 5, ncol = 5)
  - nrow ja ncol määrittävät rivien ja sarakkeiden lukumäärän
- colnames() funktiolla saa sarakkeiden nimet määritettyä
- c() komento yhdistää sisällön yhdeksi vektoriksi

paste()

```r
print(paste("Hei,", nimi)
```

4 Osa 3 - Raja-arvolause ja permutaatiotesti

Osan 3 muistiinpanot ja tehtävät.

4.1 Osa 3. alkutehtävät

Tehtävänanto: Kirjoita R-skripti, joka kysyy numeroa ja kertoo onko numero positiinen vai negatiivinen.

```
Num <- as.numeric(readline(prompt = "Syötä numero: "))

if (Num == 0){ print("Nolla") } else {

if (Num > 0){
  print("positiivinen")
} else {
  print("negatiivinen")
}

}
```

Tehtävänanto: Kirjoita R-skripti, joka kysyy kurssin arvosanaa (0-100) #ja kertoo onko kurssin suoritus hyväksytty vain hylätty. #Opiskelijan pitää saada vähintään 50 pistettä.

```
Num <- as.numeric(readline(prompt = "Syötä kurssin arvosana 0-100: "))

if (Num < 50){ print("Hylätty") }else { print("Hyväksytty") }
```

Tehtävänanto: Kirjoita R-skripti, joka kysyy numeroa ja kertoo onko numero parillinen vai pariton.

```
Num <- as.numeric(readline(prompt = "Syötä numero: "))

if (Num %% 2 ==0){ print("Parillinen") } else { print("Pariton") }
```

#parillisuus saadaan tsekattua jakojäännöksellä, jos 2 jaettaessa nolla -> parillinen # jakojäännös lasketaan %% operaattorilla

10 %% 2 10 %% 3 11 %% 2

#esim. 10/2 jakojäännös on 0 ; 10/3 jakojäännös 1 ; 11 / 2 jakojäännös 1

Tehtävänanto: Kirjoita for-silmukka, joka tulostaa numerot 1-10.

```
for (i in 1:10){ print(i) }
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
```

Tehtävänanto: Kirjoita for-silmukka, joka tulostaa lukujen 1:10 neliön.

```
for (i in 1:10){ print(i * i ) }
```

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
[1] 36
[1] 49
[1] 64
[1] 81
[1] 100
```

```
for (i in 1:10){ print(i^2 ) }
```

```
[1] 1
[1] 4
[1] 9
```

```
[1] 16
[1] 25
[1] 36
[1] 49
[1] 64
[1] 81
[1] 100
```

```r
for (i in 1:10){ print(i**2 ) }
```

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
[1] 36
[1] 49
[1] 64
[1] 81
[1] 100
```

Kolme eri tapaa

Tehtävänanto: Kirjoita for-silmukka, joka laskee lukujen 1:100 summan.

```r
summa <- 0

for (i in 1:100){ summa <- summa + i }
print(summa)
```

```
[1] 5050
```

4.2 Permutaatiotesti

4.2.1 Teoriaa permutaatiosta

Permutaatio tarkoittaa järjestystä tai uudelleenjärjestelyä

- Esim. lukujoukko (1,2,3) tai kirjainsarja (A, B, C) voivat saada 6 eri permutaatiota eli uniikkia järjestystä

- Mahdollisten permutaatioden määrä lasketaan käyttämällä kertomaa (merkitään !)

P(n)=n!

- missä P tarkoittaa permutaatioden määrää ja n alkioiden lukumäärää
- Esim. kun 4 alkiota lasketaan 4! = 4 * 3 * 2 * 1 = 24
- Tai kun 3 alkiota 3! = 3 * 2 * 1 = 6

Kertoma lasketaan R-kielellä factorial() funktiolla

```
factorial(4)
```

```
[1] 24
```

Permutaatiotesti on ei-parametrinen eli toimii millä tahansa aineistoilla

- Datan ei tarvitse siis täyttää normaalisuusoletusta

Permutaatiotestillä saadaan laskettua p-arvo eli kuinka todennäköisesti sattumalta saadaan sama tulos.

Permutaatioiden määrää lisäämällä sadaan p-arvo laskettua tarkemmin. Viljamin videoilla käytettiin 1000 tai 10 000 permutaatiota.

Thulin (2024) kirjassa neuvotaan nyrkkisääntönä käyttämään 9 999 permutaatiota, koska tällöin p arvo ainakaan usein ei asetu tasan 0,05 vaan sen alle tai yli

4.2.2 Permutaatiotesti ja bootstrap

Permutaatio tarkoittaa yhtä lukujoukon tai R-kontekstissa vektorin järjestystä. Kun ne järjestetään uudelleen, saadaan toinen permutaatio. Esimerkiksi lukujoukolla (1,2,3) on yhteensä kuusi eri permutaatiota. Tilastotieteessä permutaatiota käytetään etenkin permutaatiotestissä, jonka avulla voidaan määrittää p-arvo. Permutaatiotesti on siitä, hyvä työkalu, että se ei oleta aineiston olevan normaalisti jakautunutta, eli sitä voidaan käyttää käytännössä millä hyvänsä aineistolla.

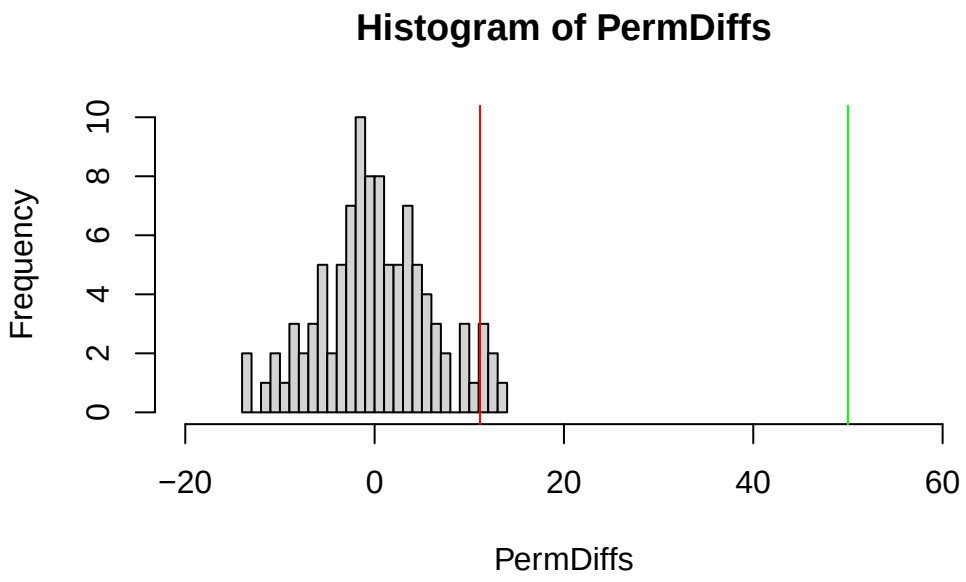
Esimerkiksi, jos kahdelta ryhmältä on mitattu koesuoritus ja halutaan tietää, onko ryhmien suoritusten välillä tilastollisesti merkitsevä ero, niin ensiksi lasketaan suorituspisteiden keskiarvojen erotus. Sen jälkeen sekoitetaan ryhmätunnisteita siten, että ryhmät jakautuvat esim. 9 999 uuteen permutaatioon ja lasketaan jokaisen permutaation kohdalla ryhmien keskiarvoero. Sitten verrataan, kuinka moni permutaatioiden keskiarvoeroista oli yhtä suuria tai suurempia kuin mitattu ero. Mikäli alle 5 % permutaatioista oli suurempia tai yhtä suuria kuin mitattu ero, niin p-arvo on alle 0,05 eli tilastollisesti merkitsevä ja nollahypoteesi voidaan hylätä.

Bootsrap on hiukan vastaava menetelmä kuin permutaatiotesti, mutta se eroaa siten, että siinä sekoitusvaiheessa alkiot arvotaan satunnaisesti uuteen ”otokseen” siten, että osa saattaa esiintyä useamman kerran ja osa ei ollenkaan. Eli kun permutaatiotestissä jokaisessa permutaatiossa on kaikki alkiot mukana, mutta vain eri järjestyksessä, niin bootstrapissä on hiukan eri kokoelma alkiota valikoituneena alkuperäisestä otoksesta. Yhdessä ”otoksessa” jokin alkio saattaa siis esiintyä vaikkapa kolme kertaa ja jokin ei kertaakaan. Samalla tavalla kuin permutaatiotestissä tuo satunnaistus toistetaan esim. 10 000 kertaa ja bootstrapissä ideana on arvioida kuinka hyvin alkuperäinen otos kuvaa perusjoukkoa, jota uudet satunnaisesti luodut otokset ikään kuin kuvaavat. Sen myötä bootstrapillä voidaan arvioida luottamusvälejä eli sitä, kuinka todennäköisesti mitattu suure osuu määritellylle välille. Yleensä luottamusvälit arvioidaan 95% tasolle, joka tarkoittaa, että jos mittaus toistettaisiin 100 kertaa niin 95 kertana luottamusvälit arvioitaisiin oikein ja perusjoukon todellinen arvo olisi niiden sisällä ja 5 kertaa taas ulkopuolella.

Permutaatiotesti: Sekoittaa ryhmätunnisteet (luo nollahypoteesin rikkomalla yhteyden tiedon ja ryhmän välillä).

```
#permutaatiotesti

# tehdään kokeilumielessä aineisto, jossa ryhmät selkeästi eroavat toisistaan
group1 <- 1:50
group2 <- 51:100

# luodaan dataframe, jossa value on ryhmien arvot ja rep() funktiolla tehdään labelit rymhmil
data <- data.frame(value = c(group1, group2), group = rep(c(1,2), each=50))


# lasketaan ryhmien keskiarvojen ero

#         yksinkertaisesti GrpDiff <-  mean(group2) - mean(group1)

# Viljamin parempi tyyli, jolla saadaan haettua tieto dataframesta myöhemmminkin


GrpDiff <- mean( data$value[ data$group==2]) - mean( data$value[ data$group==1])

#Luodaan datasta permutaatioita ja lasketaan permutaatioiden keskiarvoerot

Nperms = 100;

# alustetaan numeric() funktiolla muuttujaan oikea määrä paikkoja permutaatioille

PermDiffs <- numeric(Nperms)
```

```r
# for loopissa sample funktiolla luodaan Nperms kertaa uusi permutaatio labeleille
#   eli 1 ja 2 sekoitetaan uuteen järjestykseen
# sitten lasketaan keskiarvoerot uudella permutaatiolla ja talletetaan PermDiffs muuttujaan
#   indeksoituna [i]
# tämä toistetaan Nperms kertaa


for(i in 1:Nperms){

  # shuffle labels
  #     sample funktio arpoo otoksen lukuarvot satunnaiseen järjestykseen eli tekee uuden per
  PermLabels <- sample(data$group)

  # lasketaan ryhmien keskiarvojen erotus Nperms määrälle permutaatioita

  PermDiffs[i] <- mean( data$value[ PermLabels==2]) - mean(data$value[ PermLabels==1])

}

# Histogrammista voidaan katsoa onko havaittu keskiarvoero suurempi kuin
    # 95 % satunnaisista permutaatioista

# abline() komennolla voidaan piirtää pystyviivat histogramiin
#   abline(a,b) funktiossa a = vakio eli x-akselin leikkauskohta, b = kulmakerroin
  #   pystysuoran viivan voi piirtää abline(v=a) komennolla, v = vertical tässä kontekstissa
# todellinen keskiarvoero saadaan piirrettyä lisäämällä se abline(v= ) komentoon

# kohta, jossa 95% permutaatioista on pienempiä saadaan quantile(x , 0.95) funktiolla
# se talletetaan OneSided muuttujaan

OneSided <- quantile(PermDiffs, .95)

hist(PermDiffs, breaks = 30,  xlim=c(-20,60))
abline(v=GrpDiff, col="green")
abline(v=OneSided, col="red")
```




```
#myös voidaan manuaalisesti etsiä kohta, jossa permutaatioiden keskiarvoero on sama, kuin toc
  #   ryhmien ero, ja katsoa onko kuinka iso osa permutaatioista sen alle
# sortataan permutaatiot sort() funktiolla

sort(PermDiffs)
```

```
  [1] -13.76 -13.44 -11.20 -10.20 -10.12  -9.92  -8.84  -8.28  -8.04  -7.32
 [11]  -7.20  -6.68  -6.32  -6.24  -5.76  -5.68  -5.24  -5.16  -5.16  -4.76
 [21]  -4.56  -3.68  -3.52  -3.48  -3.40  -3.04  -2.96  -2.68  -2.64  -2.64
 [31]  -2.40  -2.08  -2.00  -1.84  -1.68  -1.68  -1.60  -1.60  -1.48  -1.40
 [41]  -1.32  -1.28  -1.08  -0.96  -0.92  -0.88  -0.56  -0.52  -0.12  -0.12
 [51]  -0.12   0.04   0.04   0.08   0.08   0.24   0.36   0.40   0.96   1.28
 [61]   1.44   1.52   1.72   1.84   2.04   2.44   2.80   2.96   2.96   3.24
 [71]   3.36   3.48   3.52   3.60   3.88   4.00   4.04   4.48   4.72   4.84
 [81]   4.84   5.12   5.28   5.44   5.52   6.04   6.24   6.36   7.52   7.80
 [91]   9.28   9.56   9.76  10.40  11.12  11.44  11.84  12.20  12.32  13.24
```

```
# tässä tapauksessa kaikki permutaatioiden keskiarvoerot jäävät todellisen keskiarvoeron alle
  # mukaan ero on hyvinkin merkitsevä
  # sen vuoksi myöskään Viljamin tapa laskea p-arvo ei toimi, koska ei indeksistä ei tule yht
  # toimii silloin, kun permutaatioista vähintään 1 on yli havaitun keskiarvoeron
#p-arvon laskeminen
```

```r
#SortedDiffs <- sort(PermDiffs)
#FirstIndex <- min(which( SortedDiffs > GrpDiff))
#Pval <- 1 - FirstIndex/Nperms


# geminin ehdottama tapa laskea p-arvo

# sum() laskee TRUE-arvojen määrän (TRUE = 1, FALSE = 0)
  # eli laskee niiden kertojen lukumäärän, kun permutaatioiden ero, on suurempi kuin havaittu
Osumat <- sum(PermDiffs >= GrpDiff)
Pval <- Osumat  / Nperms

# gemini myös suositteli lisäämään 1 sekä osoittajaan, että nimittäjään, jolla vältetään
#  tasan 0 p-arvo
  # tasan 0 p-arvo on teoreettisesti mahdoton, koska alkuperäinen havainto on oikeasti yksi p

Osumat <- sum(PermDiffs >= GrpDiff)
Pval <- (Osumat + 1)/(Nperms + 1)
Pval
```

```
[1] 0.00990099
```

```r
# t.test(arvot ~ ryhmä, data = aineisto)

t.test(value ~ group, data = data)
```

```
    Welch Two Sample t-test

data:  value by group
t = -17.15, df = 98, p-value < 2.2e-16
alternative hypothesis: true difference in means between group 1 and group 2 is not equal to
95 percent confidence interval:
 -55.78567 -44.21433
sample estimates:
mean in group 1 mean in group 2
           25.5            75.5
```

```r
t.test(group1, group2)
```

```
    Welch Two Sample t-test

data:  group1 and group2
t = -17.15, df = 98, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -55.78567 -44.21433
sample estimates:
mean of x mean of y
     25.5      75.5
```

5 Osa 4 - Tekoälyn käyttö ohjelmoinnin tukena

Tässä osassa asiaa luennoista, joissa käsitellään tekoälyn käyttöä ohjelmoinnissa. Lisäksi osan 4 tehtävien muistiinpanot.

5.1 Ohjelmointia tekoälyn avustamana, luento 26.03

5.1.1 AI:n käyttö

- koodin kirjoittaminen
- debuggaaminen ja koodin arviointi
- koodin selittäminen
- koodin optimointi

5.1.2 AI-työkaluja

- Yliopiston suosittelemat (tietoturvan vuoksi, etenkin currechat)
  - Currechat
  - Copilot

5.1.3 R-koodin tuottaminen

- yksityiskohtaiset promptit
- englanti voi toimia paremmin kuin suomi
- tarkat tilastolliset termit
  - esim. reverse coding items/variables
- kerro mitkä ovat data framen/tiedoston nimi & kerro muutttujien nimet
  - skripti toimii suoraan

- Ole tarkkana, että skripti tekee kaiken minkä pyydät
- ole skeptinen, AI tekee oikopolkuja
- jos vähänkin epäilet, niin pyydä tarkennusta
- Kehitä skriptiä iteratiivisesti – pyydä lisää ominaisuuksia / tarkennuksia; pyydä vaihtoehtoisia tapoja ratkaista asia
- Jos virhe, syötä se AI:lle ja pyydä korjaamaan
- Tee tehtävät huolellisesti
  - jos et itse ymmärrä niin todnäk ovat virheellisiä

5.1.4 Skriptien optimointi ja organisointi

- For-loop voi olla hidas, löytyy muitakin mahdollisia toteutuksia esim. replicate/apply
  - ei vakava ongelma, koska laskentatehoa riittää
  - kannattaa tehdä skriptejä, joita itse ymmärtää
- kommentoi tarkasti
- Tee funktioita
  - silloin voi soveltaa muihinkin aineistoihin
  - automatisoi mahd. paljon
- esim. viimeistele visualisoinnit
- kerro tekoälylle suoraan muuttujien nimet, data-frame nimi yms. niin koodi toimii suoraan
- Tee erillisiä skriptejä
  - yksi, joka tekee preprosessoinnin
  - toinen, joka tekee tilastoanalyysit
  - kolmas, joka tekee visualisoinnit
  - ja pääskripti, jossa tehdään kaikki/tarvittavat vaiheet oikeassa järjestyksessä

Salmela (2026)

5.2 PreProsessointi funktio tehtävä

Tehtävänanto:

> *Tee funktio, jolle voi antaa 5 argumenttia: data-kehikon, muuttujat joita ei käännetä, muuttujat jotka käännetään (reverse coding), muuttujien minimin ja maksimin.*
>
> *Funktio sitten muuntaa asteikon ulkopuolella olevat arvot puuttuviksi havainnoiksi, laskee summamuuttujan, tarkistaa poikkeavat havainnot, tekee tilastotestin summamuuttujan normaalijakautuneisuudelle, raportoi sen tuloksen selväsanaisesti (onko vai ei, mikä oli tulos), piirtää kuvan jossa kolme kuvaa vierekkäin: summamuuttujan histogrammi, summamuuttujan Q-Q plotin, summammuuttujaa vastaavan simuloidun normaalijakauman Q-Q plotin.*
>
> *Sovella funktiota HSQ-aineistoon (Humor Styles Questionnaire)*
>
> *palauta 2 tiedostoa, R-skripti ja pdf, jossa neljän HSQ-summamuuttujan jakaumat*

Tehtävän ratkaisu:

```
#ladataan data
library(readr)
library(dplyr)
library(ggplot2)
library(patchwork)
HSQdata <- read_csv("HSQdata.csv")

# HSQ käännettävät muuttujat 'Q1', 'Q7', 'Q9', 'Q15', 'Q16', 'Q17', 'Q22', 'Q23', 'Q25', 'Q2


affiliative <- c("Q1", "Q5", "Q9", "Q13", "Q17", "Q21", "Q25", "Q29")

selfenhancing <- c("Q2", "Q6", "Q10","Q14", "Q18", "Q22","Q26", "Q30")

aggressive <- c("Q3", "Q7", "Q11", "Q15", "Q19", "Q23", "Q27", "Q31")

selfdefeating <- c("Q4", "Q8", "Q12", "Q16", "Q20", "Q24", "Q28", "Q32")



PreProsFunktio<- function(data, variables = NULL, r_variables = NULL,  min = 1, max = 5, Suml

  # r_variables kohtaan sarakkeet, jotka käännetään
  # variables kohtaan muut
```

```r
# Tarkistetaan, onko r_variables annettu ja onko siinä sisältöä
if (!is.null(r_variables) && length(r_variables) > 0){

  data[, r_variables] <- (max + min) - data[, r_variables]
  message("Käänteiset muuttujat käännetty.")
} else {
  message("Ei käännettäviä muuttujia, ohitetaan kääntö.")
}



#merkataan asteikon ulkopuoliset arvot puuttuviksi
# katsotaan mitkä arvot ovat liian pieniä TAI liian suuria
virheelliset <- data < min | data > max

# Korvataan nämä kohdat NA-arvolla
data[virheelliset] <- NA
message("Asteikon ulkopuoliset arvot poistettu")
# tarkistetaan poikkeavat havainnot MAD-kriteerillä

# luodaan datafreimistä vektori, johon voidaan käyttääm mad() ja median() funktioita

numarvot <- as.numeric(unlist(data[ ,1:32]))


mad_value <- mad(na.omit(numarvot))
median_value <- median(na.omit(numarvot))

#Jos poikkeava havainto niin ilmoitetaan löytyi poikkeavia havaintoja, jos ei niin päinvast


threshold <- 3 * mad_value

filtered_data <- data[abs(data - median_value) >= threshold]

if(length(filtered_data) > 0){
  message("Löytyi poikkeavia arvoja")
} else {
  message("Ei poikkeavia arvoja")}

#tehdään summamuuttujat eri huumorityypeille
```

```r
  SummaMuuttuja <- rowMeans(data[ ,SumMuut], na.rm = TRUE)

  data$SummaMuuttuja <- SummaMuuttuja

ntesti <- shapiro.test(na.omit(SummaMuuttuja))

p_arvo <- as.numeric(ntesti)

if(p_arvo[1] < 0.05){
  message("Shapiro-wilk testin p < 0.05, Summamuuttuja ei siis ole normaalisti jakautunut")
}else {
  message("Shapiro-wilk testin p > 0.05, Summamuuttuja on normaalisti jakautunut")
}

# Summamuuttujaa vastaava simuloitu normaalijakauma

set.seed(123)
SimuloituJakauma <- rnorm(length(na.omit(SummaMuuttuja)), mean(SummaMuuttuja, na.rm = TRUE),

Simdata <- data.frame(SimuloituJakauma)

# luodaan muuuttuja, johon tulee vain annetun summamuuttujan nimi
Labs <- deparse(substitute(SumMuut))

# Piirretään kuvaajat ggplotilla, qqplot, simuloidun jakauman qqplot ja histogrammi
 p1 <- ggplot(data, aes(sample = SummaMuuttuja )) +
    geom_qq() +
    geom_qq_line(color = "red") +
    labs(title = paste(Otsikko, "QQ-plot")) +
    theme_minimal()

 p2 <- ggplot(Simdata, aes( sample = SimuloituJakauma )) +
   geom_qq() +
   geom_qq_line(color = "red") +
   labs(title = "Simuloidun jakauman QQ-plot") +
   theme_minimal()



 p3 <- ggplot(data, aes( x = SummaMuuttuja)) +
           geom_histogram(fill = "lightblue", color = "black", bins = 10) +
           labs(title = paste(Otsikko, "jakauma"), x = "Keskiarvo", y = "Frekvenssi") +
```

```
            theme_minimal()
  print(p1 + p2 + p3)

}




PreProsFunktio(HSQdata,  , c("Q1", "Q7", "Q9", "Q15", "Q16", "Q17", "Q22", "Q23", "Q25", "Q29


#Piirretään kuvaajat yhteeen pdf tiedostoon

# luodaan lista, johon on tallennettu summamuuttujat
kaikki_tyylit <- list(
  affiliative = affiliative,
  selfenhancing = selfenhancing,
  aggressive = aggressive,
  selfdefeating = selfdefeating)

# while komento varmistaa, ettei pdf:n kirjoitus ole jäänyt auki aiemmin
while (!is.null(dev.list())) dev.off()

#pdf komento avaa pdf:n kirjoituksen
pdf("Summamuuttujien_kuvaajat.pdf", width = 14, height = 5)

try({
  for (i in 1:length(kaikki_tyylit)) {


  # napataan listasta yksi huumorityyli kerrallaan
  nykyinen_setti <- kaikki_tyylit[[i]]


  PreProsFunktio(data = HSQdata, ,
                 c("Q1", "Q7", "Q9", "Q15", "Q16", "Q17", "Q22", "Q23", "Q25", "Q29", "Q31")
                 min = 1,
                 max = 5,
                 SumMuut = nykyinen_setti,
                 Otsikko = names(kaikki_tyylit)[i])
  }
})
```

```
dev.off()
# Tämä sulkee pdf:n kirjoittamisen
```

unlist()

- muuntaa datan matriisi tai dataframe muodosta takaisin vektoriksi
- esim.

```
numarvot <- as.numeric(unlist(data[ ,1:32]))
```

- unlist() komennolla pois kehikkomuodosta ja as.numeric() komennolla varmistetaan, että tallentuu numeerisena
- numarvot <- as.numeric(unlist(data[,1:32]))

! huutomerkki tarkoittaa loogista ei operaattoria, && ja if lausekkeessa

- esim.

  ```
  if (!is.null(r_variables) && length(r_variables) > 0)
  ```

- huutomerkki tarkoittaa komennossa !is.null(), sitä että kun r_variables muuttuja ei ole tyhjä niin ehto täyttyy –> ts. is not null
- && merkki tarkoittaa JA eli sitä, että molempien ehtojen täytyy täyttyä, jotta if lausekkeen ehdot toteutuvat
  - kahta && merkkiä käytetään lähinnä if lausekkeissa, muualla yhtä & merkki
- Tai puolestaan merkitään | pystyviivalla, if lausekkeessa kannattaa käyttää kahta || ja muualla yksi | riittää
  - eli merkitsee if lausekkeessa sitä, että jomman kumman ehdon täyttyminen riittää

5.3 Visualisointitehtävä

Tehtävänanto :

Oheisessa data-setissä on tutkittu lääketieteen opiskelijoiden visuaalisen asiantuntijuuden (professional vision/perceptual learning) kehittymistä osana dermatologian kurssia. Diagnoosien erottelu-kokeet tehtiin kurssin aluksi, keskellä ja lopuksi (Session:1-3, pre-mid-post). Oppimista mitattiin kolmella muuttujalla: %-oikein, reaktioaika, confidence-rating,

Tee R-skripti, joka visualisoi tulokset, eli oppimisen/muutoksen kurssin aikana (pre-mid-post) kolmen outcome muuttujan suhteen.

Tutki dataa lisää, kokeessa oli 47 eri diagnoosia. Mitkä diagnoosit käyttäytyvät samanlaisesti? Klusteroi diagnooseja yhteen ja visualisoi oppiminen. Missä diagnooseissa opiskelijat tekivät eniten virheitä? Luokittele vaikeat/helpot diagnoosit ja visualisoi oppiminen. Ideoita/esimerkkejä voi katsoa artikkeleista (data osittain eri) tai keksiä ihan uusia tapoja.

Palauta R-skripti ja erikseen kuvat pdf:nä.

```
# Skripti tulostaa PDF tiedoston, jossa on kuvaajat kaikille outcome muuttujille sessioittai
  # kuvaus siitä miten diagnooseja on jaoteltu klustereihin ja kuvaajat oppimisen kehityksest
    # klustereittain

# oppimisen muutokset kurssin aikana pre-mid-post
# 47 eri diagnoosia, mitkä käyttäytyvät samanlaisesti

# klusteroi diagnooseja yhteen ja visualisoi oppiminen
# missä tehtiin eniten virheitä
# luokittelet vaikeat ja helpot diagnoosit ja visualisoi oppiminen

# "Correct"         %-correct
# "respRT"          decision time, ms
# "Confidence"      confidence rating, 1-5

# Jokaisella on 47 vastausta per sessio ja 2 eri moduulia = 282 vastausta

library(gridExtra)
library(grid)
library(readr)
library(ggplot2)
library(patchwork)

PerceptualLearnData <- read_csv("data/PerceptualLearn/PerceptualLearnData.csv")
#haetaan data
```

```
oikeinbydiagnoosi <- aggregate(Correct ~ SelectedDiag, data = PerceptualLearnData, FUN = mean
#


keskikvartiiliväli <- oikeinbydiagnoosi$SelectedDiag[oikeinbydiagnoosi$Correct > 0.763 & oike
IQRtekstinä <- paste(keskikvartiiliväli, collapse = ", ")

vaikeimmat <- oikeinbydiagnoosi$SelectedDiag[oikeinbydiagnoosi$Correct < 0.6884807]
vaikeimmatTekst <- paste(vaikeimmat, collapse = ",")

helpoimmat <- oikeinbydiagnoosi$SelectedDiag[oikeinbydiagnoosi$Correct > 0.9159686]
helpoimmatTekst <- paste(helpoimmat, collapse = ",")

quantile(oikeinbydiagnoosi$Correct, probs = seq(0, 1, 0.1))

# 10 persentiili = 68,85 %
# 90 persentiili = 91,60 %
# keskikvartiiliväli =  76,30 % < oikein < 86,13 %

# havainnollistetaan vaikeimpien diagnoosien kehitys sessioittain
  # haetaan selectedDiag sarakkeesta vain tietyt diagnoosit ja katsotaan miten ne meni oikein
    # se onnistuu subset() funktiolla

vaikeat <- subset(PerceptualLearnData, SelectedDiag %in% vaikeimmat)
kehitysVaikeat <- aggregate(Correct ~ Session, vaikeat, FUN = mean)

# tehdään sama helpoimmille ja keskikvartiilivälille

helpot <- subset(PerceptualLearnData, SelectedDiag %in% helpoimmat)
kehitysHelpot <- aggregate(Correct ~ Session, helpot, FUN = mean)

keski <- subset(PerceptualLearnData, SelectedDiag %in% keskikvartiiliväli)
kehitysKeski <- aggregate(Correct ~ Session, data = keski, FUN = mean)


print(ggplot(PerceptualLearnData, aes(x = as.factor(SelectedDiag), y = Correct)) +
  # Piirretään keskiarvopylväät
  stat_summary(fun = "mean", geom = "bar", color = "black", width = 0.5, na.rm = TRUE) +
  # Lisätään virhejanat (Standard Error)
  stat_summary(fun.data = "mean_se", geom = "errorbar",color = "red",  width = 0.3,na.rm = TR
```

```
  labs(title = "Mittaustulokset sessioittain",
       x = "diagnoosi",
       y = "Osuus vastauksista oikein") +
  theme_minimal())


#  Määritellään sarakkeet, jotka halutaan y-akselille
y_muuttujat <- c("respRT", "Correct", "Confidence") # Vaihda nämä omiin sarakkeisiisi


#  Suljetaan mahdolliset aiemmat jumiutuneet PDF-yhteydet
while(!is.null(dev.list())) dev.off()

#  Avataan PDF:n kirjoitus
pdf("VisualisointiDiagnoosiTeht.pdf", width = 8, height = 6)

kuvaaja_lista <- list()

# for-looppi, joka piirtää kuvaajat
for (i in y_muuttujat) {

  p <- ggplot(PerceptualLearnData, aes(x = as.factor(Session), y = .data[[i]], fill = Sessio
    stat_summary(fun = "mean", geom = "bar", color = "black", width = 0.8, na.rm = TRUE) +
    stat_summary(fun.data = "mean_se", geom = "errorbar", color = "red", width = 0.3, na.rm =
    labs(title = paste("Mittaustulokset:", i),
         x = "Sessio",
         y = paste("Keskiarvo (", i, ")", sep = "")) +
    theme_minimal() +
    theme(legend.position = "none") # Piilotetaan turha selite


  kuvaaja_lista[[i]] <- p
}
otsikko_teksti <- textGrob("Visualisointitehtävä",
                           gp = gpar(fontsize = 20, fontface = "bold"))

# Tulostetaan otsikko omalle sivulleen
grid.arrange(otsikko_teksti)


#tulostetaan sessioiden mittaustulokset
wrap_plots(kuvaaja_lista, ncol = 2)
```

```r
# tulostetaan oikeinvastaamisprosentti diagnoosittain

print(ggplot(PerceptualLearnData, aes(x = as.factor(SelectedDiag), y = Correct)) +
        # Piirretään keskiarvopylväät
        stat_summary(fun = "mean", geom = "bar", color = "blue", fill = "blue", width = 0.5,
        # Lisätään virhejanat (Standard Error)
        stat_summary(fun.data = "mean_se", geom = "errorbar",color = "red",  width = 0.3,na.r

        labs(title = "Oikeiden vastausten osuus diagnooseittain",
             x = "Diagnoosi",
             y = "Osuus vastauksista oikein") +
        theme_minimal())



# Luodaan sivu, jossa kerrotaan, miten diagnooseja klusteroitiin
otsikko <- textGrob("Diagnoosien klusterointi",
                    gp = gpar(fontsize = 18, fontface = "bold"))

#  Luodaan leipäteksti
teksti_sisalto <- paste("Diagnooseja jaoteltiin kolmeen eri klusteriin:
                    \n
                    \n 1. desiliiliin ja 10. desiiliin sekä keskikvartiiliväliin oikein vasta
                    \n 1. desiiliin kuuluvat alle 10 persentiilin alle jäävät, 10. desiiliin
                    \n menevät ja keskikvartiiliväliin 1. ja 3. kvartiilin välille asettuvat
                    \n kaikista haastavimmat diagnoosit, 10. desiilissa kaikista helpoimmat j
                    \n haastavuudeltaan keskimääräiset diagnoosit.
                    \n
                    \n 10 persentiili = 68,85 % diagnooseista oikein
                    \n 90 persentiili = 91,60 % diagnooseista oikein
                    \n keskikvartiiliväli =  76,30 % < diagnooseista oikein < 86,13 %
                    \n
                    \n
                    \n 1. desiiliin kuuluvat eli vaikeimmat diagnoosit:", vaikeimmatTekst,
                    "\n\n 10. desiiliin kuuluvat eli helpoimmat diagnoosit:", helpoimmatTekst
                    "\n\n Keskikvartiilivälille kuuluvat diagnoosit:",IQRtekstinä)


leipateksti <- textGrob(teksti_sisalto,
                        x = 0.1, # Aloituskohta vasemmalta (0-1)
                        just = "left", # Tasaus vasempaan reunaan
                        gp = gpar(fontsize = 8))
```

```r
# grid.arrange asettaa ne allekkain. heights määrittää tilan jaon (otsikko vie 1/10, teksti 9
grid.arrange(otsikko, leipateksti, ncol = 1, heights = c(0.1, 0.9))

# tulostetaan oikein vastaamisen kehitys vaikeissa, helpoissa ja keskitasoisissa kysymyksissä

datasetti <- list(vaikeat, helpot, keski)
kuvaaja_lista2 <- list()
nimet <- list("Vaikeat", "Helpot", "Keskimääräiset")

for( i in 1:length(datasetti)){
  p <- ggplot(datasetti[[i]], aes(x = as.factor(Session), y = Correct, fill = Session)) +
    stat_summary(fun = "mean", geom = "bar", color = "black", width = 0.8, na.rm = TRUE) +
    stat_summary(fun.data = "mean_se", geom = "errorbar", color = "red", width = 0.3, na.rm =
    labs(title = nimet[i],
         x = "Sessio",
         y = "Oikeiden diagnoosien osuus") +
    theme_minimal() +
    theme(legend.position = "none")

  kuvaaja_lista2[[i]] <- p
}

wrap_plots(kuvaaja_lista2, ncol = 3)

# Luodaan sivu, jossa kerrotaan diagnosoinnin oppimisesta
otsikko2 <- textGrob("Oppimimisen erot klustereittain",
                     gp = gpar(fontsize = 18, fontface = "bold"))

teksti_sisalto2 <- "Kuten edellisen sivun kuvaajista nähdään, vaikeimmissa diagnooseissa
                     \n oppimisen kehitys oli suurinta ja helpommissa pienintä."

leipateksti2 <- textGrob(teksti_sisalto2,
                         x = 0.1, # Aloituskohta vasemmalta (0-1)
                         just = "left", # Tasaus vasempaan reunaan
                         gp = gpar(fontsize = 12))

grid.arrange(otsikko2, leipateksti2, ncol = 1, heights = c(0.1, 0.9))
```

```r
# Suljetaan PDF:n kirjoitus
dev.off()
```

6 Osa5 - Versionhallinta, Anova ja Regressio

Tässä osassa kahden luennon muistiinpanot.

6.1 Luento 09.04: Versionhallinta, organisointi, kommentointi

6.1.1 Versionhallinta

- dokumentoidaan skripitien versiot ja niihin tehdyt muutokset, hyötyjä:
- helpottaa yhdessä työskentelyä
- virheen tullessa voi palata edelliseen versioon
- jos tulee yhteensopivuusongelma toisen ohjelman tai paketin kanssa voidaan käyttää aiempaa versiota

Git, Gitlab, Github

- skriptit kopioituna pilvessä
- pull -> ladataan uusin versio pilvestä
- push ->päivitetään muutokset pilveen
- HY:llä käytössä gitLab

Skriptien ja datojen organisointi

- Skriptit, data, kuvat yms omissa kansioissaan
- Nimetkää järkevästi/erottuvasti
- ÄLÄ koske Rstudion tai Quarton automaattisesti tekemiin tiedostoihin.
- eivät ole turhia!
- Poistamalla tai siirtämällä saattaa aiheuttaa vahinkoa
- Ajan mittaan kertyy paljon skriptejä, dataa ja kuvia yms. joten järkevää järjestellä, jotta löytyy helposti jatkossa

Skriptien kommentointi

- Kirjoita kuvaus mitä tekee, miten toimii, mitä haluaa inputiksi, mitä tuottaa
- jokaisen rivin / osan alkuun kuvaus mitä tekee

Skriptien parantelu AI:n avulla

- AI:tä voi pyytää tekemään kommentoinnit ja kuvaukset
- Voi myös pyytää tehostamaan koodia
- Skriptistä funktio, itse tai AI:n avulla
- yhteen tehtävään spesifi skripti vs. yleisesti toimiva funktio
  - jos samantyyppistä pitää toistaa niin funktio on kätevä
  - tai skripti, jota voi muutamalla muutoksella soveltaa uuteen aineistoon
- Tekoälyä voi myös pyytää tekemään promptin, jolla voi generoida skriptin

Klusterointi

- Klusterianalyysi, cluster analysis
- kmeans() funktio
- ok myös miettiä itse perusteet klusteroinnille

Salmela (2026)

6.2 Luento 16.04: Anova & Regressio

6.2.1 Tilastotestit

Mallin valinta

- yleensä useita muuttujia -> lineaarinen regressio / lineaarinen sekamalli
- kokeellisessa tutkimuksessa perinteisesti ANOVA / toistomittausANOVA
- Korrelaatiomatriisi, jos paljon jatkuvia muuttujia
- T-testi, jos ensin tehty jokin mallinnus tms.
- esim. kontrolloitu muut vaikuttavat muuttujat, häiriömuuttujat
- Mallin / mallien rakentaminen

- Päävaikutukset
- Interaktiot
- Satunnaisvaikutukset
- astetta monimutkaisempi
- jos lineaarinen sekamalli, lniear mixed effects model
- Linkki -funktio ja residuaalien jakauma
  - vielä astetta monimutkaisempi
- jos yleistetty lineaarinen malli, generalized linear model
- dentity, log, logit …. normal, gamma, poisson
- linkkifunktiot vasta maisterivaiheessa

Yksi vai useampi malli?

- Esim mediaation testaaminen
- Lähtökohtaisesti yksinkertaisempi malli on parempi, monimutkaiset eivät välttämättä konvegoidu
- Muuttujien ja mallin rakentaminen teorian pohjalta

Mallien sovitus

- likelihood
- REML (restricted maximum likelihood)

Mallien vertailu

- malli vs. nolla-malli
- monta mallia, selitysosuus / AIC

6.2.2 1-suuntainen anova

- Testataan keskiarvoeroja silloin kun ryhmiä on enemmän kuin 2 ja voidaan ottaa huomioon muitakin muuttujia
- Varianssi aineistossa voi aiheutua
  - Ryhmien välisestä vaihtelusta
  - Ryhmien sisäisestä vaihtelusta

Varianssien suhde

- Anovassa verrataan ryhmien välistä vaihtelua ryhmien sisäiseen vaihteluun
- F = MSbetween / MSwithin
- Testataan F-testillä
- Anova taulukko
  - SS = sum of squares = neliösumma
  - MS = Mean squares = keskineliö = varianssi
  - df = degrees of freedom
  - F-arvo = anovan testisuure
  - i = henkilö
  - yms tsekkaa kaavat ja opetttele laskemaan

6.2.3 Faktoriaalinen anova

- 2 tai useampia faktoreita
  - jokaisella 2 tai useampia tasoja (level)
  - mitataan kaikki kombinaatiot
- Puretaan vaihtelu / varianssi osiin:
- Kokonaisneliösumma koostuu
  - Faktorin A (ikä) neliösumma
    - rivit
  - Faktori B (ryhmä) neliösumma
    - Sarakkeet

6.2.4 Useampisuuntainen anova

- 3 suuntaisessa 3 päävaikutusta
  - 3 kaksisuuntaista interaktiota
  - yksi kolmesuuntainen interaktio

6.2.5 ANCOVA

- kun halutaan ottaa jatkuvia muuttujia mukaan
- kovariaatiksi jokin jatkuva muuttuja
- Esim ikää tutkittaessa voidaan pitää se kategorisena tai jatkuvana
  - Ikä kategorisena
    - 2 suuntainen anova, ikä faktoriksi, interaktio mukaan
  - Ikä jatkuvana

6.2.6 ANOVAN oletukset

- Varianssit ovat saman suuruisia eli homogeenisiä
- Aineisto on normaalijakautunuttta
- virhevarianssi on normaalijakutunut
- QQ plot
- Anova suht robusti
- Sfäärisyys oletus
- jos toistomittaus niin eri tasoilla sama varianssi
- Mauchlys sphericity test
- Jos sfäärisyys oletus rikkoutuu, tehdään Greenhouse Geisser korjaus vapausasteitiin ja niiden kautta myös p-arvoon

6.2.7 ToistomittausANOVA

- usein kaikki koehenkilöt mittaavat monta tilannetta
- Anovan solut eivät enää toisistaan riippumattomia
- Toistomittaus-analyysi
- Otetaan huomioon, että sama kohenkilö tekee monta tilannetta
- tilastollinen voima parempi
- varianssi vähenee, koska sama henkilö tekee useita tehtäviä
- otetaan huomioon yksilölliset vaihtelut tiltateiden yli

R:llä suuri määrä eri paketteja Anovan tekemiseen

Nekliösumman tyyppi

- Tyypin 1 neliösumma
  - R-studio käyttää tätä oletuksena
  - järjestyksellä on väliä
- Tyypin 2 neliösuma
  - interaktio jätetään huomiomatta
- Tyypin 3 neliösumma
  - Tätä käytetään useimmiten tutkimuksessa
  - otetaan huomioon kaikki efektit
  - järjestyksellä ei väliä ja interaktio otetaan huomioon
  - kaikki efektit kontrolloituna
  - car paketti
  - Anova(res, type=3)

6.2.8 Puuttuvat arvot Anovassa

- jos yksikin arvo puuttuu anovassa koko koehenkilö hylätään
- pyri varmistamaan, että kaikilta koehenkilöiltä saadaan kaikki mittaukset
- jos yksittäinen arvo puuttuu voi korvata / imputioida keskiarvolla
- imputointi = puuttuvien arvojen korvaaminen
- jos enemmän puuttuvia arvoja voi kokeilla MICE alogoritmia
  - multiple imputation by chained equations
- Paras tapa: käytä lineaarista sekamallia, se sallii puuttuvat arvot
  - automaattisesti sallii puuttuvat arvot

6.2.9 Post Hoc testit ja kontrastit

Anovan jälkeen parittaiset vertailut -> post hoc t- testit

- P-arvojen korjaus & efektikoko
- Tukey, Bonferroni, Holm, jne.

Tai kontrastit (painotetut neliösummat)

```
-    Usein etukäteen (a priori) tideossa mikä efekti kiinnostaa

-   Testataan vain se -\> enemmän tilastollista voimaa, koska vähemmän vertailuja
```

- Voi olla lineaaeisia tai quadraattisia tai jotakin muuta

6.2.10 Anova vs GLM

- GLM = yleistetty lineaarinen malli
- voi olla mikä tahansa linkkijakauma, mikä tahansa aineisto
- Lineaarinen malli GLM:n erityistapaus, jossa oletetaan, että yhteys on lineaarinen
- Anova on lineaarisen regression erityistapaus
- usein voi tehdä anovan sijaan regression ja saa identtisen tuloksen
- Joskus anova on tulkinnallisesti helpompi ymmärtää

6.2.11 Mallit Anova & lineaarinen regressio

Mallien sovitus dataan

```
-   Anova
    -   aov()

<!-- -->

-   regressio

    -   lm()

-   lineaarinen sekamalli

    -   lmer()
```

- Mallien tarkastelu
  - summary()
  - anova()
  - library(car)
  - car::Anova(res, type=3)
  - car paketin anovassa voi päättää sen tyypin, tyyppi 3 paras
- Mallin rakentaminen
  - vainpäävaikutukset
- F arvo on t-arvo toiseen (F = t^2)
- plot() funktiolla voi piirtää sitä sun tätä
- emmeans paketilla posthoc testit
  - emmeans()
- sjstats paketilla efektikoko
  - anova_stats()
- :: kaksoispiste notaatio tarkoittaa, että haetaan tietystä paketista tietty funktio
  - esim. sjstats::anova_stats

    - sjtats paketista haetaan funktio anova_stats

Salmela (2026)

6.3 Toistettava ja avoin tiede (luentovideo)

OSF = Open Science Framework

- https://osf.io
- yksi paikka johon julkaistaan tutkimusten skriptejä ja dataa

BIDS = Brain Imaging Data Structure

- tietty tyyli, jolla dataa järjestellään neurotutkimuksessa
    - Kansioidaan kuvat
        * anat
        * fmap
        * func
- OpenNeuro
    - https://openneuro.org
    - Avoin tietokanta, josta löytyy neurokuvantamisen dataa

Salmela (2026)

6.4 Erilaisia aineistoja ja niiden erityispiirteitä (luentovideo)

Aineiston esikäsittely iso-osa tutkimustyötä ja tilastollista päättelyä

Kyselyaineisto

```
-   yleensä vain numeerista tai kategorista

-   siivous helppoa
```

Kokeellinen tutkimus

```
-   Useampia trialeita eli koekertoja -\> kymmeniä / satoja vastauksia

    -   Ei kannata ottaa kaikkea tilastotaulukkoon, vaan pitää preprosessoida

-   Saattaa mennä niin monimutkaiseksi ettei onnistu pelkästään R-studiolla, vaan vaatii eri

    -   Esim. EEG kokeissa, fMRI kokeissa, geenitesteissä yms.
```

Tekstiaineistot

```
-   Haastattelu

    -   litterointi tekstiksi

    -   laadullinen analyysi ja luokittelu tai sisältöanalyysi

-   Tekstitietokannat / corpus

-   R-studioon löytyy paketteja

    -   <https://content-analysis-with-r.com>
```

Internet / Some / keskustelupalstat

- Nykyään yleinen tapa kerätä aineistoa
  - vaatii myös eettisen ennakkoarvioinnin
  - ei voi vain suoraan vapautuneesti käyttää

Pelit / sovellukset

- Peliyhtiöt keräävät ja analysoivat
  - käyttäjien vastaukset / reagoinnit, käyttöaika yms.
    * Samoin muut sovelluskehittäjät
- Organisaation käyttämä oma sovellus / ohjelma
  - Esim. teams, kalenteri yms.

Salmela (2026)

6.5 Replicate funktio for-loopin sijaan (luentovideo)

Skriptien tehokkuus ja selkeys

- videolla korvattiin for-looppi replicate funktiolla permutaatiotesti-skriptissä
- Ihmettelin, että miksi p-arvo lasketaan Viljamin skriptissä mean() funktiolla

p -arvo laskettiin videolla seuraavalla koodilla, jossa CorrelationsPermuted muuttuja sisältää vektorin, johon on tallennettu permutaatioiden korrelaatiot ja Correlation muuttuja sisältää alkuperäisen kahden ryhmän korrelaation

```
pval <- mean(CorrelationsPermuted>=Correlation)
```

- Jos mean() funktion sisään asettaa loogisen ehdon niin se ei laskekaan annettujen numerojen keskiarvoa, vaan sen kuinka suuri osuus tapauksista täyttää ehdon

Esimerkiksi alla funktio laskee, että luvuista 1-10 50 % ovat yli 5.

```
mean(1:10>5)
```

```
[1] 0.5
```

-
- Eli koodi mean(CorrelationsPermuted>=Correlation) laskee kuinka suuri osuus permutoiduista korrelaatiosta on suurempia tai yhtäsuuria kuin alkuperäinen mitattu korrelaatio.

Se toimii laskee siis saman kuin koodinpätkä, jota käytin omassa permutaatiotestissäni

```
p_arvo <- sum(abs(PermDiffs) >= abs (ogcor)) / Npermutations
```

- Kun summafunktiolle sum() annetaan looginen ehto se laskee yhteen kuinka monta kertaa ehto täyttyy.

Esimerkiksi alla sum() funktio laskee, että ehto täyttyy 5 kertaa

```
sum(1:10>5)
```

```
[1] 5
```

6.6 Skriptistä funktio (luentovideo)

- funktio luodaan function() funktiolla
- Funktion sisältävää skriptiä kutsutaan source() komennolla
  - source() komennon jälkeen funktiota voidaan käyttää normaalisti

Salmela (2026)

6.7 Skriptien ja funktioiden kuvaus (luentovideo)

- On hyvä tapa kommentoida ja kuvata selvästi tekemiään funktioita
- Heti skriptin alkuun kannattaa kirjata otsikko, kuvaus, argumentit ja käyttö

Yksi tyyli selkeyttää otsikkoa on laittaa risuaitoja sen ympärille

```
#########################
# Otsikko
#########################

# KUVAUS
# tämä funktio tekee sitä ja tätä

# ARGUMENTIT
# mitä argumetteja funktiolle voi antaa

# KÄYTTÖ
# miten skriptiä käytetään

# TULOSTE
# mitä skripti tulostaa

# Kuka tehnyt ja milloin viimeiseksi muokattu
```

6.8 Matikka-aineiston käsittelytehtävä

Tehtävänanto:

Tutkimusavustajan tehtävät jatkuvat. Matikkatutkimuksen aineisto on kerätty ja oppilaiden täyttämät vastauslomakkeet digitoitu. Ohessa zip-tiedosto, jossa aineisto. Ohessa myös tiedot tehtävistä (tehtävän vaikeustaso, 1=helppo, 2=vaikea; oikea vastaus a-d). Tee 3 skriptiä. Yksi, joka koostaa yksittäiset tekstitiedostot data-kehikoksi, eli poimii tiedot erillisistä tiedostoista. Sitten yhdistä nämä kaksi data-kehikkoa (poimitut datat ja taustatiedot) ID:n perusteella, ja koostaa tiedoista tarvittavat muuttujat (ensin tarkistaa mitkä tehtävät oikein, sitten laskee %-oikein keskiarvon). Toinen skripti, joka tekee aineistolle ANOVAn (koulun x luokka x sukupuoli vaikutus %-oikein) ja raportoi sen tulokset ymmärrettävässä muodossa taulukoin/sanallisesti. eli kirjoita muutamalla lauseella löytyikö merkitseviä tuloksia ja mikä tuloksen aiheutti. Kolmas skripti, joka visualisoi tulokset (keskimääriin %-oikein +/- 95% luottamusvälit, koulu x luokka x sukupuoli).

Huom! Tehtävällä vastausaikaa 27.4. asti. Tehtävä kuuluu osiin 5 ja 6. Voi olla haastava tehtävä. Skripti 1, datan koostaminen yhteen usein hankalaa. Skripti 2, anova, yritä löytää merkitsevät efektit ja keksiä mistä syntyvät (kontrastit, parittaiset vertailut). Skripti 3, piirrä siistit kuvat tuloksista.

palauta 3 skirptiä ja tuloskuvat pdf.nä

Materiaalit:

```
Taustatiedot csv:  "data/MatikkaAineistonKasittelyTehtava/DataTaustaTiedot.csv"
Oppilaiden vastauksien tekstitiedostot: "data/MatikkaAineistonKasittelyTehtava/MatikkaLomakke
```

Vastaus:

6.8.1 Skripti 1

```
####################################
# Matikka-aineiston preprosessointi
####################################




library(readr)
library(tidyverse)
library(dplyr)

# Puretaan tiedostot väliaikaiseen kansioon
zip_polku <- "data/MatikkaAineistonKasittely.zip"
```

```r
purku_kansio <- "data/MatikkaLomakkeet2"
# unzip() funktiolla saa zip tiedoston purettua haluaamaansa kansioon
unzip(zip_polku, exdir = purku_kansio)

# luetaan read.csv() funktiolla taustatiedot csv tiedosto ja talletetaan se Tausta muuttujaa
Tausta <- read.csv("data/MatikkaLomakkeet2/MatikkaAineistonKasittely/DataTaustaTiedot.csv")



# Haetaan lista kaikista tekstitiedostoista
tiedostot <- list.files(purku_kansio, pattern = "\\.txt$", full.names = TRUE, recursive = TR

# Luodaan funktio, joka lukee yhden tiedoston ja poimii tarvittavat tiedot
lue_oppilas <- function(polku) {
  # Luetaan tiedoston sisältö (oletetaan, että tiedot ovat riveillä tai sarakkeissa)
  # Jos ID on esimerkiksi ensimmäisellä rivillä ja vastaukset sen alla:
  sisalto <- readLines(polku)

  # ID on rivillä 5, sukupuoli rivillä 7 ja tehtävien vastaukset riveillä 11, 13, 15, ... , 
  oppilas_id <- sisalto[5]
  sukupuoli <- sisalto[7]
  vastaukset <- sisalto[seq(11,29, by = 2)]

  # Luodaan yhden rivin data frame
  res <- data.frame(
    ID = oppilas_id,
    T1 = vastaukset[1], T2 = vastaukset[2], T3 = vastaukset[3], T4 = vastaukset[4],
    T5 = vastaukset[5], T6 = vastaukset[6], T7 = vastaukset[7], T8 = vastaukset[8],
    T9 = vastaukset[9], T10 = vastaukset[10], Sukupuoli = sukupuoli,
    stringsAsFactors = FALSE
  )
  return(res)
}
# Suoritetaan lukeminen kaikille 8400 tiedostolle lapply funktiolla
# Tämä palauttaa listan, jossa on 8400 pientä data framea
lista_tuloksista <- lapply(tiedostot, lue_oppilas)

# Yhdistetään lista yhdeksi suureksi data frameksi kaikki_data
kaikki_data <- do.call(rbind, lista_tuloksista)
```

```r
# 1. Poistetaan teksti "ID: " ja kaikki välilyönnit
# "" tarkoittaa, että korvataan löydetty teksti "tyhjällä"
kaikki_data$ID <- gsub("ID: ", "", kaikki_data$ID)
# Varmuuden vuoksi poistetaan mahdolliset reunavälilyönnit trimws() funktiolla
kaikki_data$ID <- trimws(kaikki_data$ID)

# 2. Varmistetaan vielä, että Tausta framen ID on samassa muodossa (pelkkä numero tekstinä) 
Tausta$ID <- as.character(Tausta$ID)
Tausta$ID <- trimws(Tausta$ID)


# Poistetaan tekstit kaikista vastaussarakkeista, jotta pystytään vertailemaan niitä oikeiden
sarakkeet <- c("T1", "T2", "T3", "T4", "T5", "T6", "T7", "T8", "T9", "T10", "Sukupuoli")

# lapply ajaa gsub-komennon jokaiselle listatulle sarakkeelle ja poistaa sieltä turhan teksti
kaikki_data[sarakkeet] <- lapply(kaikki_data[sarakkeet], function(x) gsub(".*: ", "", x))




# Yhdistetään Tausta ja kaikki_data yhdeksi dataframeksi siten, että se jaottelee ne ID:n mul
yhdistetty_data <- left_join(kaikki_data, Tausta, by = "ID")


# poistetaan tyhjät välilyönnit ID:n lisäksi koko muustakin datasetistä trimws() funktiolla
yhdistetty_data[, 2:35] <- lapply(yhdistetty_data[, 2:35], trimws)

# Luodaan uusi sarake 'Pisteet', joka laskee oikeat vastaukset per rivi
# Vertaa sarakkeita T1-T10 (2:11) sarakkeisiin Oikea1-Oikea10 (26:35)
yhdistetty_data$Pisteet <- rowSums(yhdistetty_data[, 2:11] == yhdistetty_data[, 26:35])

# Lasketaan onnistumisprosentti (koska tehtäviä on 10)
yhdistetty_data$Pisteet_Prt <- (yhdistetty_data$Pisteet / 10) * 100

# Katsotaan tulos
head(yhdistetty_data[, c("ID", "Pisteet", "Pisteet_Prt")])

head(yhdistetty_data)

mean(yhdistetty_data[ ,2:11] == yhdistetty_data[26:35])
```

unzip()

- unzip(zip_polku, exdir = purku_kansio)
- purkaa pakatun tiedoston haluttuun kansioon
- ekaan argumenttiin tulee purettava tiedosto ja exdir= argumenttiin kansio, johon tiedosto puretaan
- read.csv()
  - lukee .csv tiedoston ja sen voi tallettaa muuttujaan

list.files()

- tiedostot <- list.files(purku_kansio, pattern = "\\.txt$", full.names = TRUE, recursive = TRUE)
- listaa annetun kansion sisältämät tiedostot, jos ajaa ilman määrityksiä niin listaa kaikki projektikansion sisältämät tiedostot
  - Olennaisia argumenttejä Gemini (2026)

    path: Mistä kansiosta etsitään? (Esim.path = "data/lomakkeet")

    pattern: Millaisia tiedostoja etsitään? Tämä käyttää säännöllisiä lausekkeita (regex).

    Esim.pattern = "\\.txt$"etsii vain.txt-päätteisiä tiedostoja.

    full.names = TRUE: Tämä on kriittinen!

    Jos tämä onF ALSE (oletus), saat vain tiedoston nimen (esim. "lomake1.txt").

    Jos tämä on TRUE, saat koko polun ("data/lomakkeet/lomake1.txt"), jolloin voit antaa polun suoraan funktioille kuten > readLines() tai read.csv().

    recursive = TRUE: Jos kansion sisällä on alikansioita, tämä asetus käskee R:n "sukeltamaan" myös niihin ja hakemaan tiedostot kaikilta tasoilta.

regex lausekkeet

- lyhenne sanoista regular expressions eli yleiset ilmaisut
- tietynlainen hakukieli
- Voi etsiä, poimia tai korvata tekstiä sen avulla
- tiedostot <- list.files(purku_kansio, pattern = "\.txt$", full.names = TRUE, recursive = TRUE)

"\\.txt$"

- \ –> (Escape-merkki): Kun laitat pisteen eteen kenoviivan \., "kumoat" pisteen erikoistarkoituksen. Kerrot tietokoneelle: "Tarkoitan ihan oikeaa, tavallista pistettä".
- \\ –> Koodissa on kaksi kenoviivaa, koska R vaatii yhden kenoviivan "suojellakseen" toista kenoviivaa, joka taas suojelee pistettä.
- $ -> dollarimerkki tarkoittaa rivin loppua eli list.files() etsii vain tiedostoja, jotka päättyvät .txt
- saman voi tehdä ilman regex logiikkaa lisäämällä fixed = TRUE argumentin

```
# Tämä etsii kirjaimia ".txt" ilman regex-logiikkaa
list.files(pattern = ".txt", fixed = TRUE)
```

Merkki	**Merkitys**	**Esimerkki**
`.`	Mikä tahansa yksittäinen merkki (paitsi rivinvaihto).	`k.t` löytää "kat", "kot", "k-t".
`^`	Rivin alku.	`^Moi` löytää vain rivit, jotka alkavat sanalla "Moi".
`$`	Rivin loppu.	`loppu$` löytää vain rivit, jotka päättyvät sanaan "loppu".
`*`	Edellinen merkki toistuu 0 tai useita kertoja.	`ab*` löytää "a", "ab", "abbb".
`+`	Edellinen merkki toistuu 1 tai useita kertoja.	`ab+` löytää "ab", "abbb", mutta ei "a".
`[ ]`	Merkkiperhe: mikä tahansa sulkujen sisällä oleva merkki.	`[abc]` löytää a:n, b:n tai c:n.
`\\d`	Mikä tahansa numero (R-kielessä tarvitaan kaksi kenoviivaa).	`\\d+` löytää luvun, kuten "123".

Taulukko 5.1. *Regexin perusaakkosia. Taulukko on Gemini (2026) :n tekemä.*

readLines()

- sisalto <- readLines()
- lukee annetun tekstitiedoston ja tallentaa sen riveittäin vektoriksi. Eli rivi 1 tallentuu omaksi alkioksi ja rivi 2 omakseen jne. Sitten voi hakea muuttujasta indeksin mukaan halutun rivin. Esim. sisalto[3] palauttaa tekstitiedoston kolmannella rivillä olleen tekstin.

lapply()

- lista_tuloksista <- lapply(tiedostot, lue_oppilas)
  - lapply() funktio ottaa ensimmäiseen argumenttiin annetun listan (tiedostot) ja suorittaa sille toiseen argumenttiin laitetun funktion (lue_oppilas) jokaiselle listan alkiolle.
  - palauttaa listan, jossa on funktion tulokset
  - apply() funktioperheen jäsen, jonka erikoisominaisuus on, että käsittelee listoja ja palauttaa aina listan

do.call()

- kutsuu tiettyä funktiota
- kokoaa annetun aineiston. Eli esimerkiksi ottaa listan ja asettaa sen kerralla yhdelle funktiolle. Gemini (2026) kuvasi näin: "do.call() ei itse osaa toistaa asioita tai käydä läpi listoja rivi riviltä. Sen sijaan se osaa ottaa kasan laatikoita ja antaa ne kaikki kerralla yhdelle funktiolle"
- kaikki_data <- do.call(rbind, lista_tuloksista)
- tässä do.call funktio syöttää rbind funktiolle kerralla koko lista_tuloksista listan
- rbind()
- lyhenne sanoista rowbind
- Liittää annetun aineiston rivit yhteen
- 2 vaatimusta, että toimii
  - Annettujen taulukoiden sarakkeet täytyy olla nimetty samalla tavalla
  - Sarakkeita täytyy olla yhtä monta
- Esimerkki rbindista

```r
# Taulukko A
oppilas1 <- data.frame(Nimi = "Anni", Pisteet = 10)

# Taulukko B
oppilas2 <- data.frame(Nimi = "Pekka", Pisteet = 8)

# Liitetään ne päällekkäin
molemmat <- rbind(oppilas1, oppilas2)
molemmat
```

```
   Nimi Pisteet
1  Anni      10
2 Pekka       8
```

- cbind() eli columbind liittää yhteen kolumneja

```r
# Taulukko 1: Perustiedot
nimet <- data.frame(ID = c(1, 2), Nimi = c("Anni", "Pekka"))

# Taulukko 2: Tulokset
tulokset <- data.frame(Pisteet = c(10, 8), Arvosana = c("L", "M"))

# Liitetään ne vierekkäin
yhdistetty <- cbind(nimet, tulokset)
yhdistetty
```

```
  ID  Nimi Pisteet Arvosana
1  1  Anni      10        L
2  2 Pekka       8        M
```

  - cbind on siitä huono, että se ei tunnista nimiä tai ID-numeroita yms. Siksi left_join() on parempi, koska sille voi määritellä ehdon jonka mukaan se jaottelee yhdistettävät taulukot
  - Rivimäärän on oltava täsmälleen sama
  - Järjestys on ratkaiseva eli asiat jaotellaan kylmästi järjstyksen mukaan. –> siksi left_join() ““

left_join()

- Liittää kaksi dataframea yhteen
- yhdistetty_data <- left_join(kaikki_data, Tausta, by = “ID”)
- kaikki_data ja Tausta ovat datafreimejä.
- by = –> voi määritellä minkä mukaan tiedot järjestetään uuteen datafreimiin. Tässä tapauksessa yhdistettiin ID:n mukaan, jolloin tiedot asettuvat siten, että aina yhdellä rivillä on saman ID:n sisältämät tiedot
- Yhdistettävissä datafreimeissä täytyy siis olla vähintään yksi sama sarake, jonka mukaan tiedot voidaan järjestää. Kuten ID oli tehtävässä.

Suositus myös ajaa `nrows()` datakehikoiden yhdistämisen jälkeen, jotta näkee onko rivien määrä säilynyt samana.

Ominaisuus	**cbind()**	**left_join()**
Luotettavuus	Matala (perustuu sijaintiin)	Korkea (perustuu ID-tunnukseen)
Järjestys	Pakko olla sama	Järjestysksellä ei väliä
Rivin monistuminen	Ei koskaan (antaa virheen)	Voi tapahtua, jos avain ei ole uniikki
Puuttuva ID	Mahdoton käsitellä	Täyttää `NA`-arvoilla

Taulukko 5.2. Geminin tekemä taulukko…

`gsub()` funktion käyttö…

gsub()

- tulee sanoista global substitution
- etsii tietyn tekstin kohdan ja korvaa sen uudella tekstillä
- kaikki_data$ID < -gsub("ID:", "", kaikki_data$ID)
- Skriptissä korvattiin "ID:" teksti tyhjällä " " , koska yhdistettäessä left_join() funktiolla ID sarakkeiden sisältöjen täytyy täsmälisesti vastata toisiaan. Aiemmin toisessa oli vain numerosarja: esim 12345 ja toisessa ID: 12345. Poiston jälkeen molemmissa oli vain numerosarja.

trimws()

- poistaa välilyönnit
  - Tausta$ID < -trimws(Tausta$ID)
  - varmistettiin, ettei ID soluihin jäänyt välilöyntejä, jotta se varmasti täsmää toisen datafreimin ID sarakkeen kanssa
- mean(yhdistetty_data[,2:11] == yhdistetty_data[26:35])
  - laskee osuuden, jossa annettu ehto täyttyy

6.8.2 Skripti 2

Anovan oletusten testaus

- qq-plotin piirtäminen
  - plot(malli, which = 2)
    - which = 2 tarkoittaa nimenomaan Q-Q-plotia

Part III

R-komentoja

Tänne kerään kurssin mittaan kootusti R-komentoja, funktioita, koodinpätkiä eri aiheisiin liittyen.

7 Kuvaajien piirtäminen

7.1 Histogrammi

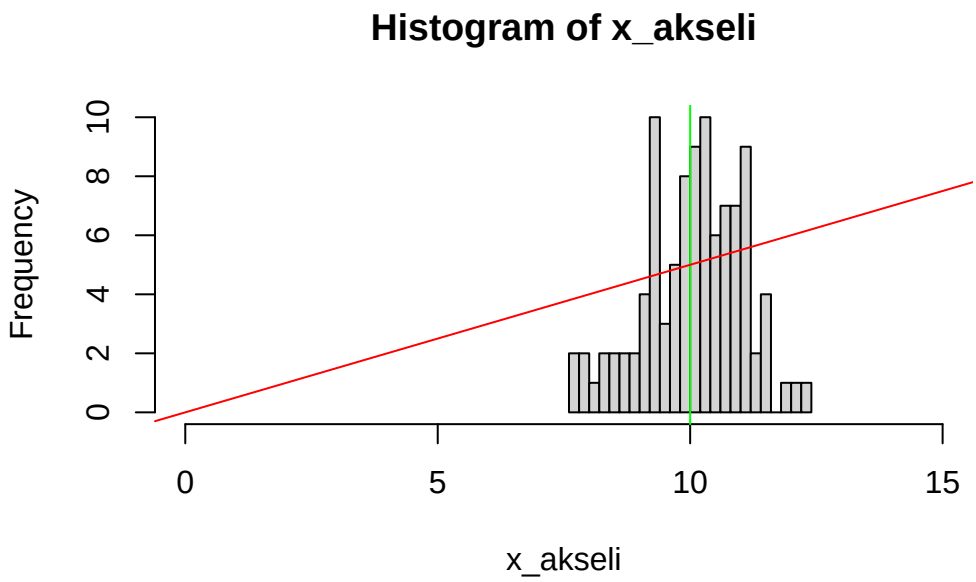
Histogrammi piirretään **hist()** funktiolla

```
hist(x , breaks = 30,  xlim=c(-20,60))
```

- x —> data, josta histogrammi piirretään
- breaks = —> voidaan määrittää millä välein palkkeja piirretään
- xlim=c(alkupiste,loppupiste) —> voidaan määritellä mikä osa x-akselista näkyy

Komennolla: **abline()** voidaan piirtää viivoja histogramiin

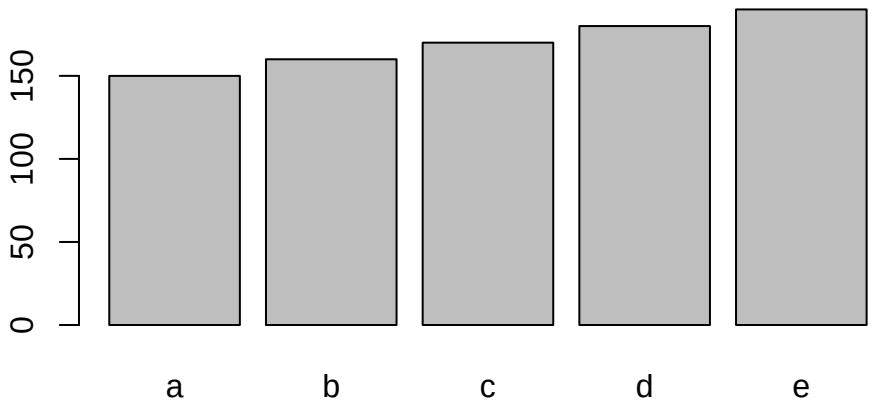
```
#luodaan aineisto
x_akseli <- rnorm(100, 10)

# piirretään histogrammi ja kaksi viivaa
hist(x_akseli, breaks = 25, xlim=c(0,15)) #
abline(v= 10, col="green")
abline(0,0.5, col="red")
```



- **abline(a,b)** funktiossa
  - a = vakio eli y-akselin leikkauskohta # (eli y:n arvo, kun x = 0),
  - b = kulmakerroin
  - pystysuoran viivan voi piirtää abline(v=a) komennolla, v = vertical tässä kontekstissa
  - vaakasuoran puolestaan abline(h=a)

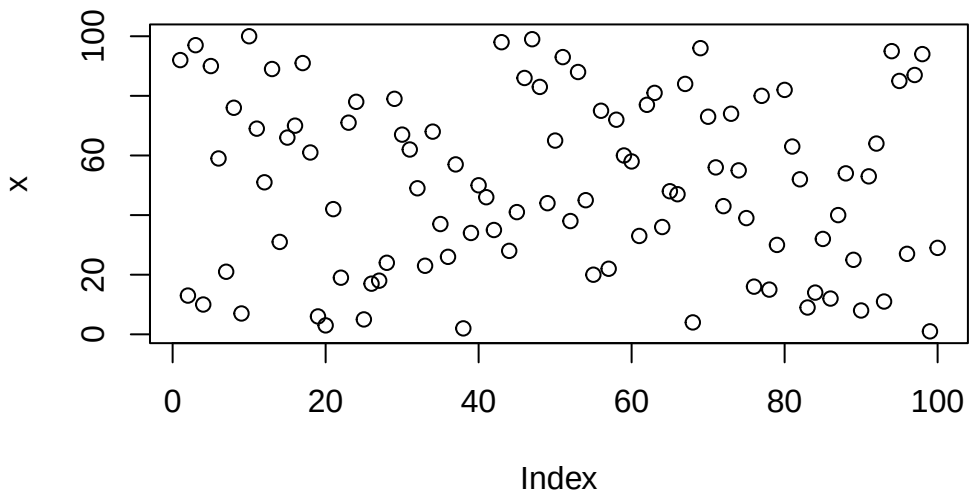
7.2 Pylväsdiagrammi

```
heights_var <- c(a=150,b= 160,c= 170,d= 180,e= 190)
barplot(heights_var)
```



7.3 Plot

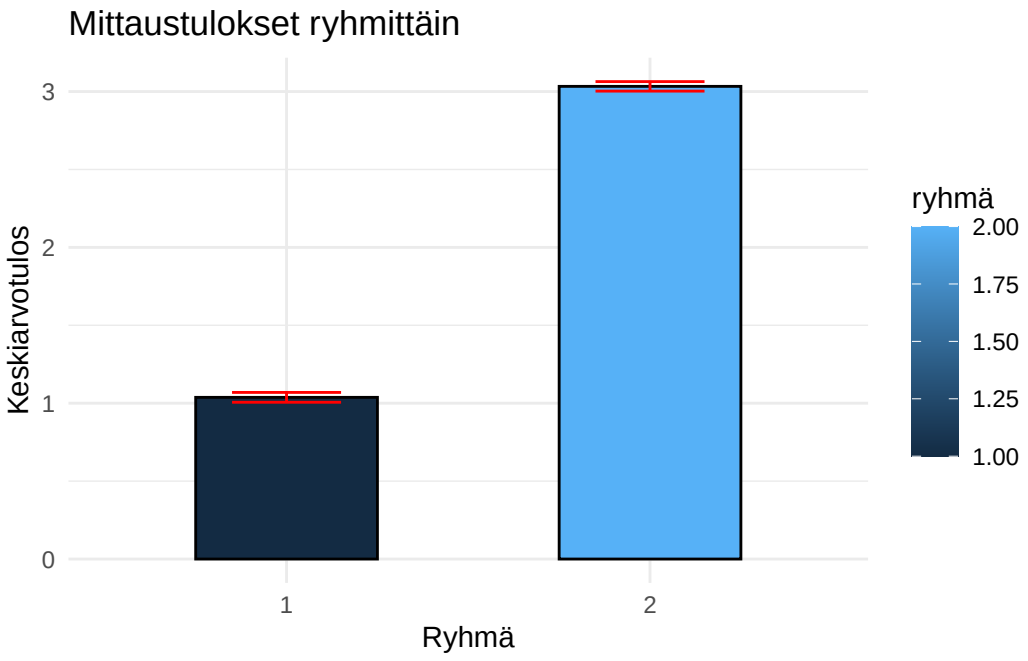
```
x <- sample(100)
plot(x)
```


x
0
20
60
100
0
20
40
60
80
100
Index

8 ggplot paketti

8.1 Pylväsdiagrammi ggplotilla

```
library(ggplot2)
R1 <- rnorm(1000,1)
R2 <- rnorm(1000,3)

data <- data.frame(arvo = c(R1,R2), ryhmä = rep(c(1,2), each=1000))

ggplot(data, aes(x = as.factor(ryhmä), y = arvo, fill = ryhmä)) +
  # Piirretään keskiarvopylväät
  stat_summary(fun = "mean", geom = "bar", color = "black", width = 0.5, na.rm = TRUE) +
  # Lisätään virhejanat (Standard Error)
  stat_summary(fun.data = "mean_se", geom = "errorbar",color = "red",  width = 0.3,na.rm = TR

  labs(title = "Mittaustulokset ryhmittäin",
       x = "Ryhmä",
       y = "Keskiarvotulos") +
    theme_minimal()
```



- ggplot(data, aes(x = ryhmä, y = arvo, fill = ryhmä))
  - data –> datasetti
  - aes(x = sarake, joka tulee x-akselille, y = sarake, joka tulee y-akselille, fill = muuttaa väriä arvoa sarakkeen alkiosta toiseen)
    - as.factor(ryhmä) –> kun merkataan as.factor saadaan tulostettua vain luvut 1 ja 2. Eli ei tule desimaaleja niiden väliin
- stat_summary(fun = "mean", geom = "bar", color = "black", width = 0.9, na.rm = TRUE)
  - fun = käytettävä funktio
  - geom = mikä muoto piirretään
  - color = väri
  - width = palkin leveys
  - na.rm = poistetaan puuttuvat arvot
- stat_summary(fun.data = "mean_se", geom = "errorbar",color = "red", width = 0.3,na.rm = TRUE)
  - luottamusvälien / virhepalkkien piirto

- labs() –> otsikot
- theme_minimal() –> teema

9 Datasetin luominen

9.1 Datan avaaminen

```
library(readr)

PerceptualLearnData <- read_csv("data/PerceptualLearn/PerceptualLearnData.csv")

View(PerceptualLearnData)
```

- readr paketista saa csv tiedostojen lukemisen komennot
- read_csv()
  - hakee tiedoston
  - tiedoston täytyy olla projektikansiossa ja mikäli se sijaitsee projektikansion alakansiossa täytyy kirjoittaa koko polku projektikansiosta eteenpäin
    * Esim. Projektikansiossa alakansio data ja sen alakansio PerceptualLearn, jossa tiedosto sijaitsee
    * kirjoitetaan "data/PerceptualLearn/PerceptualLearnData.csv"
- View()
  - näyttää haetun ja muuttujaan talletetun tiedoston

9.2 Datasetin kuvailu

aggregate() funktiolla

```
aggregate(Confidence ~ Session, data = PerceptualLearnData, FUN = mean)
```

- Voidaan laskea annettusta datasetistä asioita halutulla funktioilla ryhmitellen
- data = <- datasetti, josta tiedot haetaan

  - datasetistä PerceptualLearnData löytyy sarakkeet Confidence ja Session

- "Confidence" paikalle sarake, jossa on haluttu tieto
- "Session" paikalle ryhmittelevä muuttuja eli sarake, jonka mukaan tiedot jaetaan ryhmiin
  - Esimerkissiä oli 3 eri sessiota, jossa mitattiin itseluottamusta
  - Tuloksena tulee siis 3:n eri session itseluottamusten keskiarvot eriteltynä
- FUN = <- funktio, jotka käytetään. Esimerkissä mean() eli keskiarvo

Havainnollistava esimerkki

```
R1 <- rnorm(1000,1)
R2 <- rnorm(1000,3)

data <- data.frame(arvo = c(R1,R2), ryhmä = rep(c(1,2), each=1000))

aggregate(arvo ~ ryhmä, data = data, FUN = mean)
```

```
  ryhmä      arvo
1     1 0.9976144
2     2 2.9526546
```

- Tässä havainnollistuksen vuoksi laskettiin aggregate funktiolla keskiarvot ryhmille 1 ja 2.
- R1 keskiarvoksi määritettiin rnorm() funktiolla 1 ja R2 funktiolle 3
- aggregate() funktiolla saatiin laskettua ryhmitellyt keskiarvot

summary() funktiolla

```
summary(PerceptualLearnData$Confidence)
```

- antaa datasetin minimin, 1. kvartiilin mediaanin, keskiarvon, 3. kvartiilin ja maksimin sarakkeittain
- voi valita myös vain yhden sarakkeen viittaamalla $ merkillä

names() funktiolla

```
R1 <- rnorm(5,0)
R2 <- rnorm(5,1)

data <- data.frame(arvo = c(R1,R2), ryhmä = rep(c(1,2), each=5))

names(data)
```

```
[1] "arvo"  "ryhmä"
```

- palauttaa datasetin sarakkeiden nimet

9.3 rnorm()

```r
rnorm(n, mean, sd)
```

- luo normaalijakaumaa noudattavan vektorin, jossa
  - n —> lukumäärä
  - mean —> keskiarvo (oletuksena 0)
  - sd —> keskihajonta (oletuksena 1)
- Esim. alla luodaan normaalijakaumaa noudattava 10 alkion vektori, jonka keskiarvo on 6 ja keskihajonta 2.

```r
rnorm(10, 6, 2)
```

```
 [1] 5.152293 7.401415 6.463570 2.167912 6.334135 4.448650 5.300138 5.276036
 [9] 6.874334 7.851608
```

Tarkistetaan, että keskiarvo ja keskihajonta pitävät paikkansa.

```r
vekt10 <- rnorm(10,6,2)
vekt100 <- rnorm(100,6,2)
vekt1000 <- rnorm(1000,6,2)

m1 <- mean(vekt10)
sd1 <- sd(vekt10)
m2 <- mean(vekt100)
sd2 <- sd(vekt100)
m3 <- mean(vekt1000)
sd3 <- sd(vekt1000)
RivNimet <- c("Keskiarvo", "Keskihajonta")

data.frame(N10 = c(m1,sd1), N100 = c(m2,sd2), N1000 = c(m3, sd3), row.names = RivNimet)
```

```
                  N10     N100    N1000
Keskiarvo    5.126921 5.899459 6.162570
Keskihajonta 1.057808 2.139202 2.088264
```

Isommalla n:llä näyttäisi osuvan vielä tarkemmin kohdalleen.

9.4 set.seed() funktio

```r
set.seed(123)
```

- Määrittää mitä satunnaislukuja käytetään
- laitetaan (yleensä) vain kerran skriptin alkuun
- Hyöty on se, että määrittää joka kerta samat satunnaisluvut eli jos ajaa saman skriptin kaksi kertaa molemmilla kerroilla laskelmiin tulee samat luvut ja kuvaajat näyttävät samanlaisilta

9.4.1 Havainnollistava esimerkki

```r
set.seed(123)

Alkupiste <- rnorm(1)
Jatkopiste <- rnorm(1)

data.frame("Alku" = Alkupiste, "Jatkopiste" = Jatkopiste)
```

```
        Alku Jatkopiste
1 -0.5604756 -0.2301775
```

- Ensimmäinen luotu satunnaisluku eli Alkupiste on set.seed(123) listan ensimmäinen luku
- Toinen luotu satunnaisluku eli Jatkopiste on set.seed(123) listan toinen luku

```r
set.seed(123)
alkupiste <- rnorm(1)

set.seed(123)
uusialkupiste <- rnorm(1)

data.frame("Alku" = alkupiste, "UusiAlku" = uusialkupiste)
```

```
        Alku   UusiAlku
1 -0.5604756 -0.5604756
```

- Ensimmäinen satunnaisluku set.seed(123) listan ensimmäinen luku
- Koska set.seed(123) laitettiin skriptiin uudelleen se nollaa listan ja sitä lähdetään käymään uudelleen läpi
  - Siksi toinen luotu satunnaisluku eli UusiAlku on sama

9.4.1.1 Ero rnorm() funktiossa set.seed() kanssa ja ilman

Set.seedin() kanssa rnorm() luo siis satunnaisluvut tietystä listasta ja ne toistuvat jokaisella ajokerralla samalla tavalla. Ilman set.seed() funktiota jokaisella ajokerralla rnorm() luo eri satunnaisluvut.

Set.seedin kanssa

```
set.seed(123)
AinaSamat <- numeric(10)
for( i in 1:10){
AinaSamat[i] <- rnorm(1)
}
AinaSamat
```

Ilman set.seedia

```
AinaErit <- numeric(10)
for( i in 1:10){
  AinaErit[i] <- rnorm(1)
}
AinaErit
```

Yllä voi kokeilla ajaa koodinpätkät niiin monta kertaa kuin haluaa. Set.seedin kanssa jokaisella ajokerralla tulee samat luvut, kun ilman sitä joka kerta tulee eri luvut.

9.5 Data framen luominen

```r
R1 <- rnorm(5,0)
R2 <- rnorm(5,1)

data <- data.frame(arvo = c(R1,R2), ryhmä = rep(c(1,2), each=5))
data
```

```
          arvo ryhmä
1  -0.23017749     1
2   1.55870831     1
3   0.07050839     1
4   0.12928774     1
5   1.71506499     1
6   1.46091621     2
7  -0.26506123     2
8   0.31314715     2
9   0.55433803     2
10  2.22408180     2
```

data.frame(sarake1, sarake2, sarake3, …)

- Eli sulkuihin erotellaan pilkuilla eri sarakkeisiin menevä data
- sarakkeet pystyy nimeämään laittamalla arvojen eteen nimen ja = (esim. arvo=R1)
- esimerkissä käytetty c() eli combine funktiota, joka yhdistää kahdesta vektorista yhden
  - c() funktiolla saa siis yhdistettyä useamman vektorin yhden sarakkeen alle
- row.names= argumentilla saa rivit nimettyä
  - täytyy viitata vektoriin esim. ylemmässä keskiarvo ja keskihajonta tehtävässä luotiin kirjainvektori RivNimet, johon sitten viitattiin
    - RivNimet <- c("Keskiarvo" , "Keskihajonta")

9.6 Satunnaistaminen tai satunnaisnumero sample() funktiolla

```r
sample(x, size, replace = FALSE, prob = NULL)
```

- x -> datajoukko, josta numerot vedetään
  - esim. 1:10, generoi numeroita väliltä 1-10

    - voi myös viitata muuttujaan
- size -> montako numeroa otetaan
    - esim 1 palauttaa yhden lukuarvon ja 2 2 lukuarvoa jne
- replace = FALSE -> palautetaanko jo nostetut arvot koriin, jolloin niitä voidaan käyttää uudelleen vai käytetäänko jokaista arvoa maksimissaan kerran
    - oletuksena FALSE eli arvoja käytetään vain kerran
        * esim. permutaatiotestissä joka arvoa käyteetään vain kerran
    - jos vaihtaa TRUE niin arvoja voidaan käyttää useampaan otteeseen
        * bootstrap menetelmässä arvoja puolestaan käytetään useampaan otteeseseen

9.7 Uuden kansion luominen dir.create()

```
dir.create("uusi_kansio")
```

- Luo uuden kansion työskentelykansioon
    - työskentelykansion voi aina tarkistaa getwd() funktiolla

```
dir.create("kurssi/viikko1", recursive = TRUE)
```

- recursive = TRUE -> mahdollistaa yhdellä kerralla kansion ja alikansion luomisen
    - esimerkissä luo kansion kurssi ja siihen alikansion viikko1 kansion

Hyvä muistaa

- Ei ylikirjoita olemassaolevien kansioiden päälle, vaan antaa varoituksen
- varmista, että nojaviivat ovat eteenpäin, jos ei toimi
    - vain yksi kenoviiva kansioiden välille

9.8 Kansion luominen if-lauseen kanssa

```r
#| eval: false
uusi_paakansio <- "Koulutehtava"

if (!dir.exists(uusi_paakansio)) {
  dir.create(uusi_paakansio)
}
```

- Kansio voi olla järkevää luoda if lausekkeen kanssa koska
  - se yrittää luoda kansion, vain jos sen nimistä kansiota ei vielä ole olemassa
  - Eli sen jälkeen, kun kansio on jo luotu, niin seuraavilla ajokerroilla ei yritetä turhaan luoda sitä uudestaan
    - olemassa olevalle kansiolle ei käy mitään uusilla ajokerroilla, vaikka sitä pyrittäisiin luomaan uudestaan, mutta se antaa virheilmoituksen/varoituksen konsoliin, joka tekee siitä sekavamman
- ! <- merkki tarkoittaa loogista ei operaattoria
  - eli if lauseen ehto menee: jos uusi_paakansio nimistä kansiota ei ole olemassa niin tee seuraava asia
    - luo uusi kansio dir.create() funktiolla
  - ilman ! merkkiä ehto menisi siis: jos uusi_paakansio on olemassa niin tee seuraava asia

9.9 Tiedosto ja kansiopolun muuttaminen file.rename()

```r
file.rename(from, to)
```

- file.rename() funktiossa
  - from kohtaan alkuperäinen siirrettävän kansion nimi
  - to kohtaan uuden yläkansion nimi / siirrettävän kansion nimi

```r
dir.create("kurssi/viikko1", recursive = TRUE)
dir.create("Salatut_elamat")

file.rename("kurssi","Salatut_elamat/kurssi")
```

Esim.

- Luotiin kansio kurssi ja sen alakansio viikko1
- Lisäksi luotiin kansio Salatut_elamat
- file.rename() funktiolla saadaan siirrettyä kurssi kansio Salatut_elamat kansion alle
  - koko "kurssi" kansio siirtyy eli sen alla oleva viikko1 siirtyy samalla

9.10 Tiedostopolun kirjoittaminen file.path() funktiolla

```
file.path("tietokone", "käyttäjä", "omatkansiot", "data", "measures")
```

Kun kirjoittaa file.path() funktioon esimerkin mukaisesti R rakentaa sen näin:

1. Ottaa sanan tietokone
2. Lisää erottimen /
3. Ottaa sanan käyttäjä
4. Lisää erottimen / jne.

- Eli file.path() funktiolla saa tiedostopolun helposti talteen ilman virheitä ja kenoviivojen kanssa sähläämistä

9.11 Tiedostojen hakeminen ylä- tai alakansioista

..

- kaksi pistettä tiedostonimessä tarkoittaa, että siirrytään yksi kansiotaso ylöspäin

```
data <- read_csv("../data/PerceptualLearnData.csv")
```

- esimerkissä skripti on pääkansion alakansiossa ja haluttiin siirtyä yläkansioon ja sieltä toiseen alakansioon nimeltä data, jossa PerceptualLearnData.csv sijaitsee.

10 Tilastotestejä

10.1 T-testi

Testaa onko kahden ryhmän keskiarvojen välinen ero tilastollisesti merkitsevä.

10.1.1 T-testi syöttämällä suoraan kaksi ryhmää

```
R1 <- rnorm(50,0)
R2 <- rnorm(50,1)

t.test(R1, R2)
```

```
    Welch Two Sample t-test

data:  R1 and R2
t = -4.464, df = 96.726, p-value = 2.178e-05
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -1.3005298 -0.4999855
sample estimates:
  mean of x   mean of y
-0.07808073  0.82217692
```

Mitä tietoja testi näyttää

- t-arvo: montako keskivirheen mittaa ryhmien keskiarvot eroavat
- df = degrees of freedom = vapausasteet
- p-value eli p-arvo
- 95 percent confidence interval = luottamusväli eli Väli, jolla ryhmien todellinen keskiarvojen erotus on 95 % todennäköisyydellä.

  - eli 95 kertaa sadasta keskiarvojen erotus osuu välille ja 5 kertaa sadasta ulkopuolelle

- sample estimates: ryhmien omat keskiarvot

10.1.2 T-testi kaavamuodossa (tällöin voidaan hakea tieto dataframesta)

```
R1 <- rnorm(50,0)
R2 <- rnorm(50,1)

data <- data.frame(arvo = c(R1,R2), ryhmä = rep(c(1,2), each=50))

t.test(arvo ~ ryhmä, data = data)
```

```
    Welch Two Sample t-test

data:  arvo by ryhmä
t = -6.6722, df = 97.991, p-value = 1.511e-09
alternative hypothesis: true difference in means between group 1 and group 2 is not equal to
95 percent confidence interval:
 -1.7148355 -0.9286151
sample estimates:
mean in group 1 mean in group 2
     -0.1842226       1.1375027
```

10.2 Korrelaatio

```
x <- c(1, 2, 3, 4, 5)
y <- c(2, 4, 5, 4, 6)

cor(x, y)
```

```
[1] 0.8528029
```

cor(x,y)

- laskee x:n ja y:n korrelaation, jos ne ovat vektoreita

  - jos matriiseja niin laskee sarakkeiden x:n ja y:n sarakkeiden välisen korrelaation
- oletuksena pearsonin korrelaatio
  - voi muuttaa spearmanin tai kendallin korrelaatioon argumentilla method =
    - method = “spearman”, method = “kendall”, method = “pearson”
- `use = "pairwise.complete"` NA arvot jätetään huomioimatta (Thulin (2024))

11 Matemaattisia funktioita

11.1 Kertoma: factorial()

```
factorial(3)
```

```
[1] 6
```

```
factorial(4)
```

```
[1] 24
```

```
factorial(10)
```

```
[1] 3628800
```

Laskee kertoman (!) eli sen kuinka moneen eri järjestykseen tietty määrä lukuja voidaan laittaa

- 3 lukua –> 6 järjestystä
- 4 lukua -> 24 järjestystä
- 10 lukua -> 3 628 800 järjestystä

11.2 Kombinaatiot: choose()

```
# choose(n, k)
choose(4, 2)
```

```
[1] 6
```

Kuinka monella eri tavalla voimme valita k alkiot n alkion joukosta, kun järjestyksellä ei ole väliä?

1. Kaava

Binomikertoimen kaava perustuu kertomiin (!):

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Eli montako eri tapaa on valita tietty joukko (k) isommasta joukosta (n).

Esimerkki: 4 osallistujasta valitaan 2 hengen tiimi ($\binom{4}{2}$)

- $n = 4$ (kaikki osallistujat)
- $k = 2$ (valittavat)

$$\frac{4!}{2!(4-2)!} = \frac{4 \times 3 \times 2 \times 1}{(2 \times 1) \times (2 \times 1)} = \frac{24}{4} = 6$$

(Vaihtoehdot: {1,2}, {1,3}, {1,4}, {2,3}, {2,4}, {3,4})

Permutaatiotestissä lasketaan nimenomaan kombinaatioita

11.3 Vektorien laskutoimitukset

Vektori ja yksittäinen arvo

- jokainen vektorin arvo käydään läpi ja tehdään sille sama laskutoimitus. Esim. alla vektorin jokaisesta arvosta vähennetään 3 ja palautetaan indeksin joka kohdan tulos.

```
c(2,4,6,8,10) - 3
```

```
[1] -1  1  3  5  7
```

Kaksi vektoria

- Kahdella yhtä pitkällä vektorilla voi tehdä laskutoimituksia keskenään
- Keskenään lasketaan aina saman indeksinumeron arvo
  - Jos lasketaan esimerkikis V1 + V2, jossa V1 = c(1,2,3) ja V2 = c(2,4,6)
    * lasketaan 1. V1[1] + V2[1] = 1 + 2 = 3
    * 2. V1[2] + V2[2] = 2 + 4 = 6
    * 3. V1[3] + V2[3] = 3 + 6 = 9

```r
c(1,2,3) + c(2,4,6)
```

```
[1] 3 6 9
```

11.4 Summafunktiot: sum(), colSums() ja rowSums()

sum()

- laskee annetun aineiston kaikkien arvojen summan
- Esim dataframessa siis jokaisen solun summa eli kaikki rivit ja kaikki sarakkeet

colSums()

- laskee annetun aineiston kolumnien summat
- eli, jos datafreimissä on 20 saraketta ja 10 riviä, niin antaa vastauksena 20 summaa
  - laskee siis sarakkeen 1. 10 arvoa yhteen, sarakkeen 2. 10 arvoa yhteen, sarakkeen 3. 10 arvoa yhteen jne.

rowSums()

- laskee annetun aineiston rivien summat

  jos datafreimissä 20 saraketta ja 10 riviä, niin antaa vastauksena 10 summaa
  - laskee rivin 1. 20 arvoa yhteen, rivin 2. 20 arvoa yhteen, rivin 3. 20 arvoa yhteen jne.

nrow()

- kertoo montako riviä on annetussa aineistossa

ncol()

- kertoo montako saraketta on annetussa aineistossa

11.5 Matemaattisia operaattoreita

- `abs(x)`: laskee itseisarvon|x|.
- `sqrt(x)`: laskee neliöjuuren √x.
- `log(x)`: laskee logaritmin x jossa luonnollinen luku e on perustana.
- `log(x, base = a)`: laskee logaritmin x jossa luku a on perustana.
- `a^x`: laskee a potenssiin x
- `exp(x)`: laskee e potenssiin x
- `sin(x)`:laskee sin(x).
- `sum(x)`: kun `x` on vektori x=(x1,x2,x3,…,xn), laskee summan sen elementeistä `x`: ni=1xi.
- `prod(x)`: kun `x` on vektori x=(x1,x2,x3,…,xn), computes the product of the elements of `x`: ni=1xi.
- `pi`: a built-in variable with value , the ratio of the circumference of a circle to its diameter.
- `x %% a`: computes x modulo a.
- `factorial(x)`: laskee kertoman x!.
- `choose(n,k)`: laskee kombinaatiot (nk).

  Selitykset ovat suoraan Modern statistics with R kirjasta (Thulin (2024)).

12 Tekstikomentoja

12.1 Tekstin kirjoittaminen ja .txt tekstitiedoston luominen

```
teksti <- c("Tehtävämoniste 1 - 4 luokkalaisille",
            "Ratkaise seuraavat 10 tehtävää:",
            "",
            "Tehtävä 1:",
            "")

writeLines(teksti, "matikkatehtavat.txt")
```

- Jokainen vektorin alkio menee **writeLines()** funktiolla tallennettuun tiedostoon omalle riville
  - eli pilkulla erottamalla saa tehtyä rivinvaihdon
  - teksti kohtaan vektori, josta teksti / sisältö haetaan ja pilkun jälkeen uuden tiedoston nimi
  - tyhjän rivin saa tehtyä merkkaamalla vektoriin alkioksi pelkät hipsut “ ” , kuten riveillä 3 ja 5

12.2 Merkkijonojen yhdistäminen Paste() funktiolla

```
tallennettu <- 120

paste<- paste("jotakin", "jotakin", tallennettu)
paste0 <- paste0("jotakin", "jotakin", tallennettu)

print(paste)
```

```
[1] "jotakin jotakin 120"
```

```r
print(paste0)
```

```
[1] "jotakinjotakin120"
```

- paste funktiolla saa siis yhdistettyä useita tekstijonoja yhdeksi tekstijonoksi
  - **paste()** laittaa oletksena välilyönnin joukkojen väliin
  - **paste0()** ei laita välilyöntiä väleihin
  -

Voi myös hyödyntää esimerkiksi jos for loopilla luo juoksevan nimeämisen tiedostoille. Alla olevassa esimerkissä siirretään kansioita uuteen kansioon. For loopin avulla saadaan haettua kaikki Koulu(1-7) nimiset kansiot. Siinä siis paste0() funktiolla yhdistetään "Koulu ja juokseva numero yhdeksi kirjainjonoksi Koulu1, Koulu2, Koulu3 jne.

```r
for (i in 1:7) {
  vanha_nimi <- paste0("Koulu", i)
  uusi_nimi <- file.path(uusi_paakansio, vanha_nimi)

  # file.rename siirtää koko kansion sisältöineen päivineen
  file.rename(from = vanha_nimi, to = uusi_nimi)
}
```

12.3 PDF tiedoston luominen ja siihen asioiden kirjoittaminen

PDF luodaan komennolla **pdf()**

```r
pdf("VisualisointiDiagnoosiTeht.pdf", width = 8, height = 6)
```

- tämän jälkeisille riveille laitetun koodin tulokset tulostuvat kyseiseen pdf tiedostoon

PDF:n kirjoitus täytyy sulkea komennolla **dev.off()**

```r
# Suljetaan PDF:n kirjoitus
dev.off()
```

Ennen **pdf()** komentoa voi olla järkevää laittaa while looppi, joka varmistaa, että kaikki aiemmat PDF yhteydet ovat sulkeutuneet.

```r
#  Suljetaan mahdolliset aiemmat jumiutuneet PDF-yhteydet
while(!is.null(dev.list())) dev.off()
```

gridExtra ja **grid** pakettiin kuuluvilla komennoilla saadaan kirjoitettua tekstiä PDF:n

```r
library(gridExtra)
library(grid)

otsikko2 <- textGrob("Oppimimisen erot klustereittain",
                     gp = gpar(fontsize = 18, fontface = "bold"))

teksti_sisalto2 <- "Kuten edellisen sivun kuvaajista nähdään, vaikeimmissa diagnooseissa
                     \n oppimisen kehitys oli suurinta ja helpommissa pienintä."

leipateksti2 <- textGrob(teksti_sisalto2,
                         x = 0.1, # Aloituskohta vasemmalta (0-1)
                         just = "left", # Tasaus vasempaan reunaan
                         gp = gpar(fontsize = 12))

grid.arrange(otsikko2, leipateksti2, ncol = 1, heights = c(0.1, 0.9))
```

- **textGrob()**
  - ekaan väliin sisällytettävä teksti
  - gp = gpar(fontsize = , fontface =)
    - tekstin muotoilut
  - oletuksena piirtää tekstin keskelle (x = 0.5)
    - jos merkataan x = 0.1 niin aloittaa tekstin vasemmasta reunasta
  - just = "left"
    - tasataan teksti vasempaan reunaan
- **grid.arrange()**
  - tällä piirretään textGrob() funktiolla muotoillut muuttujat PDF tiedostoon
  - Syötetään kirjoitettavat muuttujat pilkuilla eroteltuina
  - ncol = 1 -> kertoo sarakkeiden määrän, kun on yksi niin teksti tulee sivulle normaalisti yhteen sarakkeeseen

- heights() -> määrittää muuttujien viemän tilan y-akselin suunnassa. Eli toisin sanottuna korkeuden.
    - Esim. jos heights(0.1, 0.9)
        - Ensimmäinen muuttuja vie 10 % y-akselista ja toinen 90%

print() komennolla voidaan tulostaa kuva PDF tiedostoon

```
print(ggplot(PerceptualLearnData, aes(x = as.factor(SelectedDiag), y = Correct)) +
        # Piirretään keskiarvopylväät
        stat_summary(fun = "mean", geom = "bar", color = "blue", fill = "blue", width = 0.5,
        # Lisätään virhejanat (Standard Error)
        stat_summary(fun.data = "mean_se", geom = "errorbar",color = "red",  width = 0.3,na.r

        labs(title = "Oikeiden vastausten osuus diagnooseittain",
             x = "Diagnoosi",
             y = "Osuus vastauksista oikein") +
        theme_minimal())
```

Jos halutaan tulostaa useampi samankaltainen kuva hiukan muuttuvilla parametreillä voidaan käyttää for-looppia

```
library(ggplot2)
library(patchwork)

datasetti <- list(vaikeat, helpot, keski)
kuvaaja_lista2 <- list()
nimet <- list("Vaikeat", "Helpot", "Keskimääräiset")

for( i in 1:length(datasetti)){
  p <- ggplot(datasetti[[i]], aes(x = as.factor(Session), y = Correct, fill = Session)) +
    stat_summary(fun = "mean", geom = "bar", color = "black", width = 0.8, na.rm = TRUE) +
    stat_summary(fun.data = "mean_se", geom = "errorbar", color = "red", width = 0.3, na.rm =
    labs(title = nimet[i],
         x = "Sessio",
         y = "Oikeiden diagnoosien osuus") +
    theme_minimal() +
    theme(legend.position = "none")

  kuvaaja_lista2[[i]] <- p
}

wrap_plots(kuvaaja_lista2, ncol = 3)
```

- kuvaaja_lista2 <- list()
  - **list()** komennolla alustetaan kuvaaja_lista2 muuttuja listamuotoiseksi, jolloin sinne voidaan tallettaa asioita listana
- datasetti muuttujaan talletetaan kolme hiukan erilaista data.framea listana **list()** komennolla
- nimet muuttujaan puolestaan datasettien nimet
- for-looppi suorittaa 1 -> datasetin pituus kertoja (tässä tapauksessa 3, eli 1:3) sen sisältämän koodin siten, että i tilalle asetetaan in jälkeen annetut arvot
  - eli ensimmäisellä kierroksella i = 1
  - toisella i = 2
  - kolmannella i = 3
- datasetti[[i]]
  - kaksilla hakasulkeilla viitattuna saadaan listasta datasetti haettua indeksinumerolla muuttuja
- p muuttujaan talletetaan aina luotu kuva ja se puolestaan talletetaan kuvaaja_lista2:n
  - kuvaaja_lista2[[i]] <- p
- wrap_plots(kuvaaja_lista2, ncol = 3)
  - wrap_plots() funktiolla piirretään kuvaajat
    - kuuluu patchwork pakettiin
  - ncol= määrittää sarakkeiden määrän. Jos se on kolme niin yhdelle riville tulee kolme kuvaa vierekkäin
  -

13 Varmuuskopiointi

13.1 Git ja github

```
usethis::use_git()
```

Luo olemassa olevalle R-projektille git repositorion

- uutta projektia luodessa voi rastittaa saman heti alkuun
- git repositorioon voi tallentaa varmuuskopion aina kun haluaa
  - se tehdään yläoikealta git välilehdeltä valitsemalla tehdyt muutokset ja painamalla commit

Kun on luonut tunnukset GitHubiin voi projektin tallettaa tililleen pilveen. Paikallinen git täytyy ensin yhdistää githubiin luomalla github token ja lisäämällä se Rstudioon. Github token toimii salasanana. Github suosittelee, että token on voimassa vain määräajan esim. 30 tai 90 päivää, mutta laitoin sen jatkuvasti voimassa olevaksi, koska en halua säätää sen kanssa koko ajan. Alla skripti, jolla pääsee githubin sivuille luomaan tokenin ja sen jälkeen lisäämään sen rstudioon. Token tarvitsee lisätä vain yhteen projektiin rstudiossa ja se toimii sen jälkeen kaikissa. Lisätäkseen projektin gitiin ja githubiin täytyy ajaa komennot **usethis::use_git()** ja **usethis::use_github()**

```
if(!require(usethis)) install.packages("usethis")
library(usethis)

# Tämä avaa selaimesi suoraan GitHubin oikealle sivulle
create_github_token()

if(!require(gitcreds)) install.packages("gitcreds")
library(gitcreds)

# Aja tämä ja liitä kopioimasi token (koodi) pyydettäessä konsoliin
gitcreds_set()
```

```r
# Tämä komento hoitaa kaiken: luo GitHubiin uuden kansion (repository)
# ja lähettää tiedostosi sinne.
usethis::use_github()


# Kun luo uuden projektin niin alla olevat komennot täytyy ajaa, jotta git ja github toimivat
# usethis::use_git() ja usethis::use_github().
```

13.2 .gitignore

- Gemini suositteli lisäämään alla olevat asiat gitignore tiedostoon
  - sieltä määritetään, mitä ei varmuuskopioida
  - Turhia asioita ei kannata varmuuskopioida, jotta git pysyy selkeänä ja sulavana

```
.Rproj.user
.Rhistory
.RData
.httr-oauth
.DS_Store
.quarto
/.quarto/
**/*.quarto_ipynb
_book/
_site/
site_libs/
```

14 Indeksointi / subsetting

Kun käyttää hakasulkeita [], antaa R-kielelle ohjeen poimia tietyn osan datasta. Toisin sanottuna indeksoinnilla saa poimittua mistä vain datasta osajoukon. Indeksointia on kahta päätyyppiä:

14.1 Numeerinen indeksointi

Kerrotaan suoraan havainnon järjestysnumero.

data[1] Poimii indeksin ensimmäisen arvon

```
data <- c(2,4,6,8,10)

data[1]
```

```
[1] 2
```

data[1:10] poimii indeksin arvot 1-10

```
data <- c(seq(0,100, by = 5))
data[1:10]
```

```
 [1]  0  5 10 15 20 25 30 35 40 45
```

data[15:length(data)]

- poimii indeksin arvot 15 alkaen loppuun asti
- length(data) palauttaa siis indeksin päätepisteen

```
data <- c(seq(0,100, by = 5))
data[15:length(data)]
```

```
[1]  70  75  80  85  90  95 100
```

data[length(data)]

- poimii vain indeksin viimeisen arvon eli 10

```
data <- c(2,4,6,8,10)
data[length(data)]
```

```
[1] 10
```

-

Itsekseen **length()** näyttää montako montako arvoa vektoriin kuuluu

```
data <- c(2,4,6,8,10)

length(data)
```

```
[1] 5
```

Negatiiviset indeksit palauttavat koko vektorin lukuunottamatta näitä indeksejä

- alla 1. ja 2. indeksi jäävät pois

```
data <- c(2,4,6,8,10)

data[c(-1,-2)]
```

```
[1]  6  8 10
```

Hyvönen et al. (2018)

14.2 Looginen indeksointi

Ei anneta tarkkaa järjestysnumeroa, vaan ehto, jonka on täytyttävä.

- **data[data > 5]** poimii kaikki arvot, jotka ovat suurempia kuin 5

```
data <- c(2,4,6,8,10)
data[data > 5]
```

```
[1]  6  8 10
```

Kun laittaa hakasulkeiden sisään ehdon, kuten data > 5, R tekee taustalla näkymättömän listan, joka koostuu vain sanoista TRUE ja FALSE

- Toimii käytännössä näin:
  - R käy jokaisen luvun läpi ja antaa sille leiman:
    - Luku on ehdon sisällä –> TRUE
    - luku on ehdon ulkopuolella -> FALSE
  - komento palauttaa kaikki TRUE arvot

Loogisen indeksoinnin voi tehdä myös **subset()** funktiolla

```
data <- c(2,4,6,8,10)

subset(data, data > 5)
```

```
[1]  6  8 10
```

subset(data, ehto)

- data kohtaan muuttuja, josta halutaan osajoukko
- ehto kohtaan ehto, jonka mukaan osajoukko määritellään.

%in% operaattori

- "kuuluu joukkoon" operaattori
- tarkistaa löytyykö annetusta aineistosta arvot ja palauttaa ne TRUE ja FALSE arvoina

```r
vaikeimmat <- oikeinbydiagnoosi$SelectedDiag[oikeinbydiagnoosi$Correct < 0.6884807]

vaikeat <- subset(PerceptualLearnData, SelectedDiag %in% vaikeimmat)

aggregate(Correct ~ Session, data = vaikeat, FUN = mean)
```

- koodi tarkistaa PerceptualLearnData datasetin sarakkeen SelectedDiag sisältä mitkä kaikki ovat vastaavia kuin vaikeimmat muuttujaan määritellyt arvot. Sitten subset funktio tallettaa uuden datasetin, jossa SelectedDiag sarakkeessa on vain vaikeimmat ehdon täyttäneet arvot.

Ei kuulu joukkoon eli !%in%

- huutomerkillä (!) eli looogisella ei operaattorilla voi muuntaa kuuluu joukkoon operaattorista %in% ei kuulu joukkoon operaattorin !%in%

14.3 Kaksiulotteinen indeksi

Indeksointi sana tulee siitä, että jokaisella tiedonpalasella on oma "osoitteensa" eli indeksi.

- Yksiulotteisessa datassa vain yksi indeksi: data[rivi].
- Kaksiulotteisessa datassa (kuten taulukko tai data.frame) on kaksi indeksiä: data[rivi, sarake]

14.4 Yhdet vs. kahdet hakasulkeet

Yksinkertaiset hakasulkeet []

- Yksittäiset sulkeet palauttavat aina saman tyyppisen olion kuin alkuperäinen on, mutta pienempänä. Jos leikkaat palan listasta, saat **pienemmän listan**.
  - Tulos: Alilista (sub-list).

Tuplahakasulkeet [[]] (Sisältö)

- Tuplahakasulkeet "murtavat" rakenteen ja hakevat sieltä pelkän sisällön.

15 Funktion luominen ja kutsuminen

15.1 Funktion luominen: function()

15.1.1 Perusrakenne

Funktion luominen koostuu neljästä pääosasta:

```
nimi<-function(argumentti1, argumentti2){ suoritettava koodi}
```

- Nimi -> Millä nimellä kutsut funktiota (esim. laske_keskiarvo)
- Argumentit -> Mitä tietoa funktio tarvitsee toimiakseen
  - argumentit toimivat "paikan pitäjinä" eli funktiota käytettäessä niihin syötetään mitä tahansa (sopivaa) tietoa ja esim. argumentti1 kohtaan kirjoitettu tieto menee koodissa joka kohtaan, jossa lukee argumentti1
- Runko aaltosulkeiden sisään { } –> Mitä koodia suoritetaan.
- Tulostettava arvo –> viimeinen rivi, jonka aaltosulkeiden sisällä oleva koodi tuottaa

Esimerkkinä tehdään funktio, joka laskee keskiarvon

```
laske_keskiarvo <- function(argumentti1){

  sum(argumentti1)/length(argumentti1)
  }

laske_keskiarvo(c(2,4,6,8,10))
```

```
[1] 6
```

- aaltosulkeiden sisällä olevassa koodissa argumentti1 esiintyy kahdessa paikassa. Kun funktiota käytetään niin se sijoittaa argumentti1 paikalle syötetyn tiedon koodin joka paikkaan, jossa lukee argumentti1
  - esim jos syöttäisi näin: laske_keskiarvo(c(2,2))

  - koodi funktion sisällä näyttäisi tältä:
    * {sum(c(2,2))/length(c(2,2)) }

```
laske_keskiarvo <- function(argumentti1){

  sum(argumentti1)/length(argumentti1)
  }

laske_keskiarvo(c(2,2))
```

```
[1] 2
```

15.1.2 Argumentit

Argumenttejä voi lisätä niin monta kuin haluaa ja kaikki ne samalla tavalla toimivat "paikan pitäjinä"

- Niiden järjestyksellä on merkitystä sillä funktiota käytettäessä luvut / muuttujat syötetään järjestyksen mukaan. Esim. jos on tehty useita() niminen funktio, jossa on kolme argumenttia
  - useita <- function(argumetti1, argumentti2, argumentti3){ koodi }
  - niin käytettäessä funktiota sulkeisiin ensimmäiseen väliin tuleva arvo korvaa ottaa argumentti1:n paikan, toiseen väliin argumentti2:n ja kolmanteen väliin argumentti3:n paikan.
    * useita(10, 15, 20)
      · tällöin funktio sijoittaa aaltosulkeissa olevaan koodiin jokaiseen kohtaan, jossa lukee argumentti1 arvon 10, argumentti2 arvon 15 ja argumentti3 arvon 20.
- Argumentteihin voi viitata myös niiden nimellä ja = merkillä. Sillon syöttämisen järjestyksellä ei ole merkitystä.
  - Esim useita(argumentti1 = 20, argumentti3 = 50)
  - tällöin funktio sijoittaa arvon 20 argumentti1:n paikalle ja arvon 50 argumentti3:n paikalle.

16 Sekalaista

16.1 Run-napin luominen quarto-kirjaan

Quarto-kirjaan saa luotua koodinpätkiä, joita käyttäjä pystyy html muodossa tulostuvassa kirjassa ajamaan run napilla sekä muokkaamaan. PDF muodossa näkyy vain koodi ilman sen tuottamia tuloksia. Jos palautus pitää tehdä pdf muodossa kannattaa muuttaa koodinpätkien muoto takaisin oletukseksi, jotta myös tulokset tulostuvat kirjaan.

Run napin ja interaktiivisten koodinpätkien luomiseen täytyy tehdä seuraavat asiat:

- Ajetaan terminaalissa komento: quarto add coatless/quarto-webr

  ```
  quarto add coatless/quarto-webr
  ```

  - asentaa projektiin webr lisäyksen, joka mahdollistaa interakiivisten chunkkien luomisen
  - tämä täytyy tehdä vain kerran kyseisessä projektissa

- Sen sivun ylälaitaan, johon haluaa lisätä interaktiivisia chunkkeja merkataan YAML osiin:

  - otsikon alle filters:
  - seuraavalle riville sisennettynä - webr

    ```
    ---
    title: "Oma harjoitussivu"
    filters:
      - webr
    ---
    ```

- Siihen koodinpätkään, johon haluaa interaktiivisen run-napin yläkulmaan merkataan oletuksen eli {r} sijaan {webr-r}

  ```
  {webr-r}
  ```

Näiden vaiheiden jälkeen kyseisestä chunkista ei suoraan tulostu tuloksia sivulle, vaan ne täytyy ajaa nappia painamalla ja napilla voi ajaa ne niin moneen kertaan kuin haluaa. Lisäksi käyttäjä voi muokata chunkin koodia.

```
2+2
4/4
5+5
```

16.2 Piilotettava / foldattava koodilohko quarto-kirjaan

Koodilohkoista eli code chunkeista saa tehtyä avattavia ja suljettavia kirjoittamalla koodilohkon alkuun seuraavat asiat.

```
#| code-fold: true
#| code-summary: "Näytä koodi"
```

- Yläkulmassa saa ja täytyy olla normaalisti {r}, nämä komennot sen alle
- code-summary: —> kohtaan voi kirjoittaa haluamansa tekstin, joka näkyy napissa, josta koodinpätkän voi avata.

```
aineisto <- seq(1,1000,by=5)

mean(aineisto)
```

```
[1] 498.5
```

16.3 | ja & merkit

| merkki tarkoittaa tai

- esim. x<1 | x>5
  - tarkoittaa, että ehdon toteutuakseen x:n täytyy olla pienempi kuin 1 tai suurempi kuin 5
    - riittää siis, että toinen toteutuu ehdon täyttämiseksi

& merkki tarkoittaa ja

- esim. x>1 & x<5
  - tarkoittaa, että x:n tulee olla suurempi kuin 1 ja pienempi kuin 5
    - molempien täytyy toteutua, jotta ehto toteutuu

Part IV

Tilastotiede

Tähän osioon tulee tilastotieteen teorioita, laskukaavoja yms.

17 Poikkeavat havainnot

Aineistosta voidaan poistaa outliereita eli poikkeavia havaintoja tietyillä kriteereillä. Yksi niistä on MAD-arvo eli Median Absolute Deviation eli mediaanin absoluuttinen poikkeama.

17.1 MAD-arvo

MAD arvon laskukaava

- 1. Määritetään aineiston mediaani
- 2. Lasketaan jokaisen havaintoarvon poikkeama mediaanista itseisarvona
- 3. Määritetään poikkeamien mediaani
  - Poikkeamien mediaani on MAD-arvo

Esim. Aineiston (2, 5, 7, 9, 14, 24, 250) MAD arvo lasketaan näin ->

- Mediaani on 9
- lasketaan jokaisen arvon poikkeama mediaanista

```
abs(c(2, 5, 7, 9, 14, 24, 250) - 9)
```

```
[1]   7   4   2   0   5  15 241
```

- Määritetään poikkeamien mediaani

```
sort(c(7   ,4   ,2   ,0   ,5  ,15 ,241))
```

```
[1]   0   2   4   5   7  15 241
```

- Poikkeamien mediaani eli MAD arvo on 5
- Tyypillisesti mad-arvo kerrotaan vielä vakiolla 1,4826
  - Eli lopullinen MAD-arvo on MAD * 1,4826

17.1.1 MAD-arvon laskeminen R:ssä

- R-kielessä komento mad(x) antaa oletuksena tulokseksi:
  - MAD×1.4826
- Tämä johtuu siitä, että normaalijakaumassa tämä kerroin saa MAD-arvon vastaamaan täsmälleen perinteistä keskihajontaa. Se tekee tuloksista vertailukelpoisia. Jos haluat "puhtaan" MAD-arvon ilman tätä kerrointa, R-koodi on:mad(x, constant = 1)
- Lasketaan mad() funktiolla

```
mad()
```

Www.rdocumentation.org [Verkkosivu] (haettu 2026)

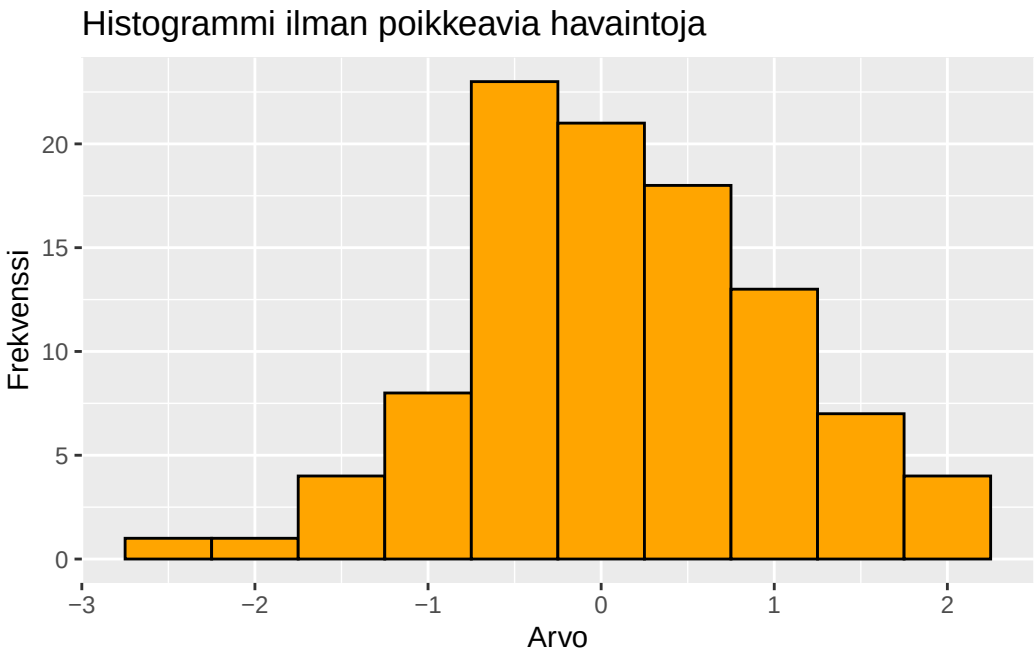
17.2 Poikkeavien havaintojen poistaminen

Alla on skripti, joka poistaa havainnosta outlierit ja piirtää histogrammin siivotusta aineistosta. Lisäksi skriptistä luodaan funktio, jolla voi tehdä saman siivouksen mille vain aineistolle.

- Poistokriteerinä käytetään MAD-arvoa, oletuksena 3 * MAD
  - käytetty R:n mad() funktiota eli vakiokerroin 1,4826 sisältyy suoraan laskuun
  - Funktiossa plot_histogram_without_outliers() neljä argumenttia
    * data —> aineisto, jota outlierit poistetaan ja histogrammi piirretään
    * kynnyskerroin —> kerroin, jolla MAD-arvo kerrotaan, oletuksena 3
    * leveys —> määrittää histogrammin palkkien leveyden, oletuksena, 1
    * palkinvari —> määrittää histogrammin palkkien värin, oletuksena sininen

```
# outliereiden poistaminen ja funktion luominen
  # skripti laskee mediaanin ja MAD arvon
    # sitten se asettaa kynnyasrvoksi 3 * MAD arvon
     # sitten luodaan muuttuja, johon tulee vain arvot, jotka ovat yhtäsuuria tai
      # pienempiä kuin kynnysarvo
        # sen jälkeen se piirtää histogrammin arvoista
          # luodaan skriptistä funktio lisäämällä alkuun function() funktio
library(ggplot2)

# luodaan esimerkkidata
```

```r
set.seed(123)
data <- rnorm(100)



#määritellään funktio

plot_histogram_without_outliers <- function(data, kynnyskerroin = 3, leveys = 1, palkinvari =

# lasketaan MAD ja mediaani

mad_value <- mad(data)
median_value <- median(data)

#Poistetaan poikkeavat havainnot
threshold <- kynnyskerroin * mad_value
filtered_data <- data[abs(data - median_value) <= threshold]
    #       abs --> muuttaa itseisarvoksi
    #       (data - median_value) --> jokaisesta datan arvosta vähennetään mediaani
      #                               eli nähdään kuinka paljon arvo eroaa mediaanista
        #     verrataan jokaisen arvon poikkemia thresholdiin eli 3 * mad arvoon
          # eli indeksoinnilla valitaan mukaan arvot, joiden poikkeama mediaanista on
            # pienempi tai yhtä suuri kuin

# piirretään histogrammi
ggplot(data.frame(x = filtered_data), aes(x)) +
  geom_histogram(binwidth = leveys , fill = palkinvari, color = "black") +
  labs(title = "Histogrammi ilman poikkeavia havaintoja",
       x = "Arvo",
       y = "Frekvenssi")
}


plot_histogram_without_outliers(data, 3 , 0.5, "orange")
```


Histogrammi ilman poikkeavia havaintoja
Frekvenssi
20
15
10
5
0
−3
−2
−1
0
1
2
Arvo

18 Normaalisuuden testaaminen

18.1 Kolmogorov-smirnov normaalisuustesti ks.test()

- Toimii paremmin suurella n:llä kuin shapiro-wilk
- R:llä toimii funktiolla **ks.test()**

18.2 Shapiro-wilk normaalisuustesti shapiro.test()

- Toimii pienellä n:llä paremmin kuin kolmogorov-smirnov
- R:ssä funktiolla **shapiro.test()**
- Samat arvot (Ties): Jos datassasi on paljon samoja arvoja (kuten Likert-asteikolla 1-5 usein on), R antaa varoituksen: "cannot compute exact p-value with ties". K-S-testi on tarkoitettu jatkuville muuttujille. Jos data on kovin epäjatkuvaa, Shapiro-Wilk (shapiro.test) on usein parempi vaihtoehto normaalisuuden testaukseen. Gemini (2026)

18.3 Normaalijakauma

$$Y_i \sim \mathbf{Normal}(\mu, \sigma)$$

You can read this as "Yi is sampled from a Normal distribution with mean and standard deviation ". So the tilde ( ) stands for "sampled from". (Speekenbrink 2023)

- Kaavassa Yi on siis yksi normaalijakauman havainto
- ~ merkki tarkoittaa, että Y on annetusta jakaumasta satunnaisesti otettu alkio
- tarkoittaa keskiarvoa
- tarkoittaa keskihajontaa

19 Matematiikan laskusääntöjä

19.1 Kymmenpotenssi (e)

- R merkkaa kymmenpotensseja tähän tyyliin
  - p-value < 2.2e-16
  - e = kymmenpotenssi
    - 2.2e-16 = 2.2 * 10^-16

      ```
      format(2.2 * 10^-16, scientific = FALSE)
      ```

      ```
      [1] "0.00000000000000022"
      ```

    - format(,scientific = FALSE) funktiolla saa esitettyä luvun desimaalimuodossa
- Kymmenpotenssissa siirretään aina pilkkua
  - 300 * 10^-2 = 3
  - 300 * 10^2 = 30 000

20 Summary

In summary, this book has no content whatsoever.

```r
1 + 1
```

```
[1] 2
```

References

Gemini, Google. 2026. *Google Gemini [Laaja Kielimalli]*. https://gemini.google.com/.

Hyvönen, Ville, Henri Karttunen, Toni Lehtonen, Aku Leivonen, and Savi Virolainen. 2018. *R-Tutuksi [Oppimateriaali]*. Helsingin yliopisto, Matematiikan ja tilastotieteen laitos. https://www.mv.helsinki.fi/home/mjxpirin/medstat_course/material/Rmoniste_tiltu1.pdf.

Navarro, Danielle, and David Foxcroft. 2025. *Learning Statistics with JAMOVI: A Tutorial for Beginners in Statistical Analysis*.

Salmela, Viljami. 2026. *SPK-3003 R-Ohjelmointi 2026 [Luentosarja]*.

Speekenbrink, M. 2023. *Statistics: Data Analysis and Modelling*. Open Book Publishers. https://mspeekenbrink.github.io/sdam-book/.

Thulin, Måns. 2024. *Modern Statistics with r: From Wrangling and Exploring Data to Inference and Predictive Modelling*. Chapman; Hall/CRC. https://www.modernstatisticswithr.com.

Www.rdocumentation.org [Verkkosivu]. haettu 2026. https://www.rdocumentation.org.